7zpus.swe@gmail.com

Norme di Progetto

Versione: 2.0.0
Responsabile: Georgescu Diana
Destinatari: Sanmarco Informatica S.p.A., Prof. Tullio Vardanega, Prof. Riccardo Cardin, Team 7-ZPUs

Tabella di Versionamento

Versione	Data	Autore	Verificatore	Descrizione
2.0.0	2026/05/01	Georgescu Diana	-	Approvazione per baseline PB
1.3.2	2026/04/06	Rocco Matteo A.	Soligo Lorenzo	Revisione e aggiunta Specifica Tecnica
1.3.1	2026/04/03	Soligo Lorenzo	Gingillino Aaron	Revisione norme
1.3.0	2026/04/02	Gingillino Aaron	Soligo Lorenzo	Aggiunta metriche di controllo
1.2.0	2026/04/01	Vigolo Davide	Fattoni Antonio	Aggiornamento sezione di infrastruttura
1.1.0	2026/03/18	Fattoni Antonio	Vigolo Davide	Aggiornamento sezione di codifica
1.0.1	2026/03/17	Laoud Zakaria	Soligo Lorenzo	Stesura paragrafo sulla progettazione
1.0.0	2026/02/24	Laoud Zakaria	Soligo Lorenzo	Approvazione per baseline RTB
0.14	2026/02/11	Vigolo Davide	Soligo Lorenzo	Definizione metriche di qualità
0.13	2026/02/09	Fattoni Antonio	Vigolo Davide	Correzione processo di documentazione e controllo della configurazione

0.12	2026/02/08	Vigolo Davide	Soligo Lorenzo	Definizione metriche di qualità
0.11	2026/02/06	Laoud Zakaria	Soligo Lorenzo	Stesura paragrafi 3.6, 3.7, 3.8
0.10	2025/01/25	Aaron Gingillino	Georgescu Diana	Scrittura paragrafi 2.2, 4.1
0.9	2025/01/25	Aaron Gingillino	Fattoni Antonio	Scrittura paragrafi 3.3, 3.4
0.8.1	2026/01/19	Soligo Lorenzo	Laoud Zakaria	Fine scrittura paragrafi 4.2, 4.3, 4.4
0.8	2026/01/11	Soligo Lorenzo	Laoud Zakaria	Stesura paragrafi 4.2, 4.3, 4.4
0.7	2025/12/15	Rocco Matteo A.	Georgescu Diana	Stesura paragrafo 5
0.6	2025/12/10	Georgescu Diana	Soligo Lorenzo	Stesura sottosezioni 2.3, 2.4
0.5	2025/12/01	Soligo Lorenzo	Rocco Matteo A.	Aggiornamento nuovo standard per gestione branch, convenzioni sui nomi, date e versioni. Sezioni 3.1.2, 3.3, 3.2.1.1.1, 3.2.4, 4.2.3.2.1
0.4.1	2025/11/28	Soligo Lorenzo	Fattoni Antonio	Correzione nella Procedura di Revisione Paragrafo 3.1.3.1 e aggiornamento immagini
0.4	2025/11/26	Soligo Lorenzo	Fattoni Antonio	Ristrutturazione completa Processi (ISO 12207: Attività-Procedure-Strumenti)
0.3	2025/11/25	Soligo Lorenzo	Fattoni Antonio	Creazione e stesura sezioni Processi di Infrastruttura e sottosezioni 4.2.1-4.3. Nuova struttura Paragrafi sottoParagrafi
0.2	2025/11/22	Soligo Lorenzo	Fattoni Antonio	Creazione e stesura sezioni Documentazione e sottosezioni 3.1-3.1.5
0.1	2025/11/16	Rocco Matteo A.	Soligo Lorenzo	Creazione e stesura sezioni Introduzione e Processo di fornitura

Indice

1 Introduzione **8**
1.1 Scopo . . . 8
1.2 Glossario . . . 9
1.3 Riferimenti . . . 9
1.3.1 Riferimenti normativi . . . 9
1.3.2 Riferimenti informativi . . . 9

2 Processi primari **10**
2.1 Processo di fornitura . . . 10
2.1.1 Attività: avvio . . . 10
2.1.1.1 Procedura: analisi approfondita del capitolato di progetto 10
2.1.1.2 Strumenti . . . 11
2.1.2 Attività: preparazione della proposta di fornitura . . . 11
2.1.2.1 Procedura: valutazione delle risorse . . . 11
2.1.2.2 Strumenti . . . 11
2.1.3 Attività: accordo . . . 12
2.1.3.1 Procedura: ridefinizione dell'accordo di fornitura . . . 12
2.1.3.2 Strumenti . . . 12
2.1.4 Attività: pianificazione . . . 12
2.1.4.1 Procedura: pianificazione delle attività . . . 12
2.1.4.2 Procedura: identificazione dei rischi . . . 13
2.1.4.3 Strumenti . . . 13
2.1.5 Attività: esecuzione e controllo . . . 14
2.1.5.1 Procedura: esecuzione del piano di progetto . . . 14
2.1.5.2 Procedura: controllo dei processi . . . 14
2.1.5.3 Strumenti . . . 14
2.1.6 Attività: revisione e validazione . . . 14
2.1.6.1 Procedura: definizione delle modalità di rendicontazione 14
2.1.6.2 Procedura: rendicontazione dello stato di avanzamento . 15
2.1.6.3 Procedura: validazione del lavoro svolto . . . 15
2.1.6.4 Strumenti . . . 15
2.1.7 Attività: consegna e terminazione . . . 15
2.1.7.1 Procedura: consegna del prodotto finale . . . 15
2.1.7.2 Procedura: esposizione delle funzionalità . . . 15
2.1.7.3 Procedura: esposizione del processo di sviluppo . . . 15
2.1.7.4 Strumenti . . . 16
2.1.8 Documentazione fornita . . . 16
2.1.8.1 Analisi dei capitolati . . . 16
2.1.8.2 Lettera di candidatura . . . 16
2.1.8.3 Lettera di presentazione . . . 16
2.1.8.4 Preventivo costi e assunzione impegni . . . 16
2.1.8.5 Verbali interni . . . 17

2.1.8.6 Verbali esterni . . . 17
2.1.8.7 Analisi dei Requisiti . . . 17
2.1.8.8 Glossario . . . 17
2.1.8.9 Piano di Progetto . . . 18
2.1.8.10 Piano di Qualifica . . . 18
2.1.8.11 Specifica Tecnica . . . 19
2.1.8.12 Strumenti . . . 19
2.1.9 Ruoli . . . 19
2.2 Processo di sviluppo . . . 20
2.2.1 Attività: analisi dei requisiti . . . 20
2.2.1.1 Procedura: raccolta e analisi dei requisiti . . . 21
2.2.1.2 Procedura: tracciamento dei requisiti . . . 21
2.2.1.3 Procedura: confronto con la committente . . . 21
2.2.1.4 Procedura: produzione del documento di Analisi dei requisiti 21
2.2.1.4.1 Casi d'uso . . . 21
2.2.1.4.2 Requisiti . . . 22
2.2.1.5 Diagrammi dei casi d'uso . . . 22
2.2.1.6 Strumenti . . . 22
2.2.2 Attività: individuazione delle tecnologie . . . 23
2.2.2.1 Procedura: individuazione delle tecnologie . . . 23
2.2.2.2 Procedura: produzione del Proof of Concept . . . 23
2.2.2.3 Strumenti . . . 23
2.2.3 Ruoli . . . 24
2.2.4 Attività: progettazione . . . 24
2.2.4.1 Procedura: progettazione architetturale (Livelli C4 1 e 2) . 24
2.2.4.2 Procedura: progettazione di dettaglio (Livelli C4 3 e 4) . . 25
2.2.4.3 Norme per la produzione dei diagrammi . . . 25
2.2.4.4 Strumenti . . . 26
2.2.5 Attività: codifica . . . 26
2.2.5.1 Procedura: definizione e applicazione delle norme di codifica . . . 26
2.2.5.2 Procedura: implementazione delle unità software . . . 27
2.2.5.3 Strumenti . . . 27
2.2.6 Attività: realizzazione dei test . . . 28
2.2.6.1 Procedura: Pianificazione dei test . . . 28
2.2.6.2 Procedura: Implementazione ed esecuzione dei test . . . 28
2.2.6.3 Procedura: analisi degli esiti e azioni correttive . . . 28
2.2.6.4 Strumenti . . . 29
2.2.7 Ruoli . . . 29

3 Processi di supporto 29
3.1 Processo di documentazione . . . 30
3.1.1 Attività: pianificazione della documentazione . . . 30
3.1.1.1 Procedura: identificazione della documentazione . . . 30

3.1.1.2 Procedura: creazione delle attività 30
3.1.1.3 Strumenti 30
3.1.2 Attività: produzione della documentazione 31
3.1.2.1 Linee guida per la produzione della documentazione . . . 31
3.1.2.1.1 Denominazione e datazione documentazione . . 32
3.1.2.2 Procedura: revisione e approvazione 32
3.1.2.3 Strumenti 32
3.1.3 Ruoli 33
3.2 Processo di Gestione della Configurazione 33
3.2.1 Attività: identificazione della configurazione 33
3.2.1.1 Procedura: identificazione dei CIs 33
3.2.1.2 Strumenti 34
3.2.2 Attività: controllo della configurazione 34
3.2.2.1 Procedura: creazione delle modifiche 34
3.2.2.2 Procedura: controllo della configurazione 36
3.2.2.3 Smart Commit 37
3.2.2.4 Standard per le Branch 38
3.2.2.5 Strumenti 38
3.2.3 Attività: registrazione delle configurazioni 38
3.2.3.1 Procedura: registrazione delle Modifiche 38
3.2.3.2 Strumenti 39
3.2.4 Ruoli 39
3.3 Processo di verifica 40
3.3.1 Analisi statica 40
3.3.2 Analisi dinamica 40
3.3.3 Attività: verifica documentale 40
3.3.3.1 Procedura: verifica del contenuto documentale 41
3.3.3.2 Procedura: verifica di correttezza grammaticale e di leggibilità 41
3.3.3.3 Procedura: verifica di aderenza alle linee guida 41
3.3.3.4 Strumenti 41
3.3.4 Attività: verifica del codice 41
3.3.4.1 Procedura: test di unità 42
3.3.4.2 Procedura: test di integrazione 42
3.3.4.3 Procedura: test end-to-end di sistema 43
3.3.4.3.1 POM pattern per i test E2E 43
3.3.4.4 Procedura: test di accettazione (Validazione) 43
3.3.4.5 Nomenclatura 43
3.3.4.6 Strumenti 44
3.3.5 Ruoli 44
3.4 Processo di validazione 44
3.4.1 Attività: pianificazione della validazione 44
3.4.1.1 Procedura: pianificazione della validazione 44
3.4.1.2 Strumenti 45

3.4.2 Attività: definizione dei test di accettazione 45
3.4.2.1 Procedura: definizione dei test di accettazione 45
3.4.2.2 Strumenti 45
3.4.3 Attività: esecuzione dei test di accettazione 45
3.4.3.1 Procedura: esecuzione dei test di accettazione 46
3.4.3.2 Strumenti 46
3.4.4 Ruoli 46
3.5 Processo di revisione congiunta 46
3.5.1 Attività: pianificazione 47
3.5.1.1 Procedura: Identificazione dei partecipanti 47
3.5.1.2 Procedura: Identificazione del materiale oggetto di revisione 47
3.5.1.3 Procedura: Pianificazione della sessione di revisione . . . 47
3.5.1.4 Strumenti 48
3.5.2 Attività: esecuzione della revisione congiunta 48
3.5.2.1 Procedure: Definizione dell'ordine di revisione del materiale 48
3.5.2.2 Procedura: revisione del materiale 48
3.5.2.3 Strumenti 48
3.5.3 Ruoli 49
3.6 Processo di garanzia della qualità 49
3.6.1 Attività: pianificazione della qualità 49
3.6.1.1 Procedura: Definizione di metriche e soglie 49
3.6.1.2 Strumenti 50
3.6.2 Attività: monitoraggio continuo 50
3.6.2.1 Procedura: monitoraggio 50
3.6.2.2 Strumenti 50
3.6.3 Attività: valutazione periodica 50
3.6.3.1 Procedura: review settimanale 50
3.6.3.2 Procedura: review di fine Sprint: 51
3.6.3.3 Procedura: review di milestone 51
3.6.3.4 Strumenti 51
3.6.4 Attività: azioni correttive 51
3.6.4.1 Procedura: Identificazione della deviazione e analisi delle cause 51
3.6.4.2 Procedura: Pianificazione dell'intervento 51
3.6.4.3 Procedura: Implementazione e verifica 51
3.6.4.4 Linee guida di gestione della qualità 52
3.6.4.5 Strumenti 52
3.6.5 Metriche di gestione della qualità 52
3.6.6 Ruoli 52

4 Processi organizzativi 53
4.1 Processo di gestione 53
4.1.1 Attività: pianificazione 53

4.1.1.1 Procedura: Identificazione 53
4.1.1.2 Strumenti 54
4.1.2 Attività: esecuzione e controllo 54
4.1.2.1 Procedura: controllo di processo 54
4.1.2.2 Strumenti 54
4.1.3 Attività: retrospettiva 54
4.1.3.1 Procedura: retrospettiva di periodo 55
4.1.3.2 Strumenti 55
4.1.4 Ruoli 55
4.2 Processo di infrastruttura 56
4.2.1 Attività: implementazione 56
4.2.1.1 Jira 56
4.2.1.2 VSCode 57
4.2.1.3 LaTeX 57
4.2.1.4 Ecosistema Google (Drive, Presentazioni, Mail) 57
4.2.1.5 Canali di Comunicazione (Discord e Whatsapp) 58
4.2.2 Attività: acquisizione degli strumenti 58
4.2.2.1 Procedura: registrazione degli account 58
4.2.2.2 Procedura: Configurazione degli strumenti 58
4.2.2.2.1 Github 58
4.2.2.2.2 Git 59
4.2.2.2.3 Jira 59
4.2.2.2.4 VSCode 59
4.2.2.2.5 LaTeX 60
4.2.2.2.6 Google Presentazioni 60
4.2.2.2.7 Google Drive 60
4.2.2.2.8 Gmail 61
4.2.2.2.9 Discord 61
4.2.2.2.10 Whatsapp 61
4.2.3 Attività: modifiche all'infrastruttura 61
4.2.3.1 Procedure di Modifica 61
4.2.3.2 Strumenti 62
4.3 Processo di formazione 62
4.3.1 Attività: pianificazione della formazione 62
4.3.1.1 Procedura: Identificazione delle necessità formative . . . 62
4.3.1.2 Procedura: Pianificazione delle attività formative 62
4.3.1.3 Strumenti 62
4.3.2 Attività: erogazione della formazione 62
4.3.2.1 Procedura: Formazione Autonoma 63
4.3.2.2 Procedura: confronto 63
4.3.2.3 Strumenti 63
4.3.3 Attività: valutazione della formazione 63
4.3.3.1 Procedura: verifica pratica 63
4.3.3.2 Strumenti 63

4.3.4 Ruoli 64

5 Metriche della qualità **64**
5.1 Qualità di Processo 65
5.1.1 Processi primari 65
5.1.1.1 Processo di fornitura 65
5.1.1.2 Processo di Sviluppo 67
5.1.1.3 Processo di Documentazione 68
5.1.1.4 Processo di Verifica 68
5.1.2 Processi Organizzativi 69
5.1.2.1 Processo di Gestione dei Rischi 69
5.1.2.2 Processo di Gestione della qualità 69
5.2 Qualità di Prodotto 69
5.2.1 Funzionalità 69
5.2.2 Affidabilità 70
5.2.3 Usabilità 70
5.2.4 Efficienza 70
5.2.5 Manutenibilità 71
5.2.6 Portabilità 72
5.3 Strumenti 72
5.3.1 Jira Software 72
5.3.2 GitHub & GitHub Actions 72
5.3.3 Strumenti di Analisi Statica e Qualità del Codice 73
5.3.3.1 SonarQube 73
5.3.3.2 Aspell & Script Python (Custom) 73
5.3.4 Strumenti di Testing e Performance (Electron-Angular) 73
5.3.4.1 Playwright 73
5.3.4.2 Electron Manager & Chrome DevTools Protocol 73
5.3.4.3 TotalValidator 74
5.3.4.4 Funzioni interne di misurazione performance 74

Elenco delle figure

1 Creazione del branch di lavoro tramite estensione Jira in VSCode 35
2 Creazione della PR verso l'issue branch *in_lavorazione* 36
3 Chiusura della PR tramite "Create Merge Commit" 37
4 Modello a V 65

1 Introduzione

1.1 Scopo

Questo documento ha l'obiettivo di definire e normare il *Way of Working*$_G$, ovvero le regole di lavoro che ogni membro del gruppo deve rispettare durante lo svolgimento

delle attività di *progetto*$_G$ volte allo sviluppo dell'applicativo software ***DIPReader***$_G$, proposto dall'azienda Sanmarco Informatica. A ciascun membro è richiesto di seguirle integralmente per poter lavorare in maniera efficace, efficiente e omogenea. Data la natura incrementale della redazione del documento, il *responsabile*$_G$ di progetto ha il compito di mantenere aggiornate le presenti norme e gli eventuali riferimenti ad altri documenti in esse contenuti.

1.2 Glossario

Ogni termine tecnico o con un significato particolare nell'ambito dell'*Ingegneria del Software*$_G$, utilizzato nella documentazione di progetto, è definito nell'apposito documento Glossario 2.0$_{(v2.0.0)}$.

1.3 Riferimenti

Il gruppo ha redatto il presente documento in conformità con lo Standard ISO/IEC 12207:1995. In alcuni punti sono stati usati anche approfondimenti tratti dalla versione ISO/IEC/IEEE 12207:2017. Questi riferimenti servono a descrivere meglio gli approcci *Agile*$_G$ e iterativi che caratterizzano lo sviluppo *software*$_G$ moderno.

1.3.1 Riferimenti normativi

- Standard ISO/IEC 12207:1995 (ultimo accesso: 20/02/2026)
- Standard ISO/IEC/IEEE 12207:2017 (ultimo accesso: 20/02/2026)
- Standard ISO/IEC/IEEE 24765:2017 (ultimo accesso: 20/02/2026)
- Capitolato C3: DIPReader (ultimo accesso: 10/02/2026)
- Regolamento di Progetto Didattico a.a. 2025/2026 (ultimo accesso: 20/02/2026)

1.3.2 Riferimenti informativi

- Dispense del corso di Ingegneria del Software 2025/2026:
  - https://www.math.unipd.it/ tullio/IS-1/2025/Dispense/T01.pdf (ultimo accesso: 20/02/2026)
  - https://www.math.unipd.it/ tullio/IS-1/2025/Dispense/T02.pdf (ultimo accesso: 20/02/2026)
  - https://www.math.unipd.it/ tullio/IS-1/2025/Dispense/T03.pdf (ultimo accesso: 20/02/2026)
  - https://www.math.unipd.it/ tullio/IS-1/2025/Dispense/T04.pdf (ultimo accesso: 20/02/2026)
  - https://www.math.unipd.it/ tullio/IS-1/2025/Dispense/T05.pdf (ultimo accesso: 20/02/2026)
  - https://www.math.unipd.it/ tullio/IS-1/2025/Dispense/T06.pdf (ultimo accesso: 20/02/2026)
  - https://www.math.unipd.it/ tullio/IS-1/2025/Dispense/T07.pdf (ultimo accesso: 20/02/2026)
  - https://www.math.unipd.it/ tullio/IS-1/2025/Dispense/T08.pdf (ultimo accesso: 17/02/2025)
  - https://www.math.unipd.it/ tullio/IS-1/2025/Dispense/T09.pdf (ultimo accesso: 17/02/2025)

  - https://www.math.unipd.it/ tullio/IS-1/2025/Dispense/T10.pdf (ultimo accesso: 17/02/2025)
  - https://www.math.unipd.it/ tullio/IS-1/2025/Dispense/T11.pdf (ultimo accesso: 17/02/2025)

- Linee Guida Sviluppo Sicuro AGID (Agenzia per l'Italia Digitale) (ultimo accesso: 22/12/2026)
- Linee Guida sulla formazione, gestione e conservazione dei documenti informatici AGID (ultimo accesso: 22/12/2026)
- Documentazione LaTeX by Lorenzo Pantieri (ultimo accesso: 17/11/2025)
- Documentazione Jira (ultimo accesso: 20/02/2026)

2 Processi primari

2.1 Processo di fornitura

Il *processo*$_G$ di fornitura contiene le attività e i compiti svolti dal *fornitore*$_G$. Questo processo è necessario per la comprensione dei *requisiti*$_G$ del prodotto da sviluppare. Per implementare correttamente il processo il gruppo si impegna a svolgere le seguenti attività.

2.1.1 Attività: avvio

Il fornitore analizza i requisiti necessari alla proposta di fornitura, tenendo in considerazione eventuali vincoli organizzativi e normativi.

2.1.1.1 Procedura: analisi approfondita del capitolato di progetto

Il fornitore deve estrarre i requisiti specificati dalla *proponente*$_G$, identificando le funzionalità richieste, i vincoli tecnici e le aspettative di qualità. Questa attività è condotta da tutti i membri del gruppo, al fine di ottenere una analisi completa e condivisa del documento. Le considerazioni fatte confluiscono nel documento di Analisi dei capitolati (v1.0.0) (§2.1.8.1). I punti da analizzare sono i seguenti:

- **Obiettivo**: identificare l'obiettivo di massima del progetto e le funzionalità richieste.
- **Dominio applicativo**: contesto di operatività del software nonché il target di utenti a cui è rivolto.
- **Dominio tecnico**: contesto tecnologico sul quale il software dovrà eventualmente integrarsi, vengono riportati i vincoli tecnologici.
- **Aspetti positivi**: identificare le opportunità formative e di crescita che il progetto può offrire, nonché i punti di forza del capitolato.
- **Aspetti negativi**: identificare le criticità e le sfide che il progetto può presentare, nonché i punti di debolezza del capitolato.

- **Possibili rischi**: identificare i rischi potenziali associati al progetto, come ad esempio la complessità tecnica, la mancanza di competenze specifiche o eventuali vincoli di tempo.

2.1.1.2 Strumenti

- **Discord:** strumento per la comunicazione asincrona e sincrona remota tra i membri del gruppo, sia essa vocale (sincrona) o testuale(asincrona).
- **LaTeX**: per la redazione del documento di Analisi dei Capitolati.

2.1.2 Attività: preparazione della proposta di fornitura

Il fornitore prepara la proposta di fornitura in risposta alle richieste della proponente e definisce i termini in cui si articola la proposta.

2.1.2.1 Procedura: valutazione delle risorse

Il fornitore deve valutare i ruoli, i costi e le risorse disponibili per soddisfare i requisiti di fornitura. Nello specifico:

- **Ruoli necessari**: identificare i ruoli necessari per lo sviluppo del progetto, come ad esempio sviluppatori, *verificatori*$_G$, analisti, ecc.
- **Costi orari**: definire i costi orari per ciascun ruolo, tenendo conto delle competenze richieste e del mercato di riferimento.
- **Stima oraria per ruolo**: per ciascun ruolo stabilito, definire una stima oraria necessaria al completamento del progetto. Questa stima deve basarsi sui risultati dell'analisi del capitolato, e fungerà da limite di spesa non superabile.
- **Componenti del gruppo di progetto**: identificare i membri del gruppo di progetto.
- **Data di consegna stimata**: definire una data di consegna stimata per il completamento del progetto tenendo conto delle risorse stanziate nei punti precedenti.

queste valutazioni confluiscono nel documento Preventivo costi e assunzione impegni (v1.0.0) (§2.1.8.4).

2.1.2.2 Strumenti

- **Discord**: strumento per la comunicazione asincrona e sincrona remota tra i membri del gruppo, sia essa vocale (sincrona) o testuale(asincrona).
- **LaTeX**: per la redazione del documento di Preventivo costi e assunzione impegni.
- **Foglio di calcolo**: per agevolare i calcoli e produrre le tabelle necessarie con facilità e garantendo correttezza.

2.1.3 Attività: accordo

Proponente e fornitore entrano nella fase di definizione dell'accordo di fornitura del prodotto software, prevedendo possibilità di negoziazione della fornitura da parte del fornitore.

2.1.3.1 Procedura: ridefinizione dell'accordo di fornitura

Il fornitore si adopera per ridefinire la proposta di fornitura, adattandola alle esigenze della proponente, se possibile. Viene iterata nuovamente l'attività di preparazione della proposta di fornitura (§2.1.2)

2.1.3.2 Strumenti

- **LaTeX**: per la redazione del documento di ridefinizione dell'accordo di fornitura.

2.1.4 Attività: pianificazione

Si pianificano le attività necessarie per soddisfare i requisiti del cliente, stabilendo tempi, risorse e costi. Viene definito il modello di sviluppo più adatto al progetto, tenendo conto delle caratteristiche del progetto e delle esigenze della proponente. È inoltre fondamentale identificare potenziali rischi che possono insorgere nello svolgimento.

2.1.4.1 Procedura: pianificazione delle attività

Il fornitore deve pianificare il lavoro a lungo termine. Deve definire *milestone*$_G$ e obiettivi intermedi. In particolare, l'identificazione delle milestone deve tenere conto dei seguenti aspetti:

- **Obiettivi di progetto**: identificare gli obiettivi principali del progetto e suddividerli in obiettivi intermedi, che rappresentano le milestone.
- **Tempi di consegna**: stabilire i tempi di consegna per ciascuna milestone, tenendo conto delle risorse disponibili e delle esigenze della proponente.
- **Risorse necessarie**: identificare le risorse necessarie per raggiungere ciascuna milestone, come ad esempio competenze specifiche, strumenti o infrastrutture.
- **Criteri di accettazione**: definire i criteri di accettazione per ciascuna milestone, in modo da garantire che il lavoro svolto soddisfi le aspettative della proponente.
- **Dipendenze tra le attività**: identificare eventuali dipendenze tra le attività necessarie per raggiungere le milestone, in modo da pianificare il lavoro in modo efficiente e evitare ritardi.

La definizione poi risulta più efficiente se programmata a ritroso a partire dalla data di consegna stimata e dalle necessità del team (date di laurea, per esempio). Avendo un limite temporale facilita la definizione delle attività da svolgere al fine di raggiungere

le milestone nelle finestre temporali stabilite. Contemporaneamente con lo svolgersi del progetto, è necessario pianificare a breve termine i periodi, definendo le attività da svolgere (vedi §4.1).

2.1.4.2 Procedura: identificazione dei rischi

Il fornitore deve valutare i potenziali *rischi*$_G$ che potrebbero insorgere durante lo sviluppo del prodotto e definire strategie di mitigazione per affrontarli efficacemente. Vanno riportati, una volta individuati, nella sezione apposita del documento Piano di Progetto (§2.1.8.9). Per ogni rischio si identificano le seguenti caratteristiche:

- **Probabilità di occorrenza**: probabilità espressa in percentuale che il rischio si verifichi durante lo svolgimento del progetto.
- **Effetti**: intesi come grado di impatto sullo svolgersi del progetto
- **Descrizione**: descrizione del rischio
- **Strategia di mitigazione**: descrizione delle azioni da intraprendere per mitigare il rischio, qualora si verificasse.
- **Indicatori**: definizione di indicatori che permettano di identificare tempestivamente l'insorgere del rischio, in modo da poter attivare le strategie di mitigazione in tempo utile.

ogni rischio deve seguire la seguente nomenclatura:

`R[T][C]`

dove:

- **T**: tipologia del rischio, compreso tra:
  - **O**: organizzativo (riguardante la gestione dei processi)
  - **T**: tecnico (riguardante le tecnologie utilizzate)
- **C**: progressivo univoco per identificare il rischio, compreso tra 01 e 99.

2.1.4.3 Strumenti

- **Jira**: per la pianificazione e il monitoraggio delle attività del progetto.
- **LaTeX**: per la redazione del documento di Piano di Progetto.
- **Foglio di calcolo**: per agevolare i calcoli e produrre le tabelle necessarie con facilità e garantendo correttezza.
- **Discord**: strumento per la comunicazione asincrona e sincrona remota tra i membri del gruppo, sia essa vocale (sincrona) o testuale(asincrona).

2.1.5 Attività: esecuzione e controllo

Il fornitore si impegna a sviluppare il prodotto secondo il Piano di Progetto, avendo cura di controllare che i processi siano stati eseguiti correttamente.

2.1.5.1 Procedura: esecuzione del piano di progetto

Il fornitore deve seguire il piano di progetto già definito, rispettando le attività pianificate, le scadenze e garantendo la qualità del lavoro svolto. Ci si aspetta quindi che i membri del gruppo si facciano responsabili del proprio lavoro e rispettino le scadenze stabilite, comunicando tempestivamente eventuali difficoltà o ritardi. È auspicabile un coinvolgimento attivo di tutti i membri del gruppo in tutte le attività di progetto, per garantire una distribuzione equa del lavoro e favorire la collaborazione.

2.1.5.2 Procedura: controllo dei processi

Il fornitore deve monitorare costantemente lo *stato di avanzamento lavori (SAL)*$_G$. Deve individuare eventuali deviazioni dal piano e adottare misure correttive per mantenere il progetto sulla giusta traiettoria. Periodicamente, tramite il cruscotto fornito da *Jira*$_G$, controlla quali attività sono state completate, quali sono in corso e quali sono in ritardo. Se necessario, rialloca le risorse liberate da *task*$_G$ completate.

2.1.5.3 Strumenti

- **LaTeX**: per la redazione del documento di Piano di Progetto.
- **Jira:** per il monitoraggio dello stato di avanzamento delle attività e l'identificazione di eventuali deviazioni dal piano.
- **GitHub:** per il versionamento del codice e della documentazione, garantendo tracciabilità delle modifiche.
- **Discord:** per la comunicazione sincrona tra i membri del gruppo in caso di difficoltà o necessità di coordinamento.
- **Cruscotto**: per il monitoraggio delle *metriche*$_G$ di processo e di prodotto, al fine di identificare eventuali criticità e adottare misure correttive tempestive.

2.1.6 Attività: revisione e validazione

Il fornitore stabilisce regolarmente un processo di revisione di quanto realizzato, garantendo feedback immediato sull'avanzamento.

2.1.6.1 Procedura: definizione delle modalità di rendicontazione

Il fornitore concorda con la proponente le modalità e i tempi di comunicazione dello stato di avanzamento del progetto, attraverso incontri periodici ed eventuali messaggi

asincroni di aggiornamento. E' buona prassi stabilire fin dall'inizio una cadenza regolare per gli incontri. La cadenza può non essere rispettata, anticipando o posticipando gli incontri a seconda della disponibilità di materiale da revisionare o di eventuali difficoltà incontrate durante lo sviluppo.

2.1.6.2 Procedura: rendicontazione dello stato di avanzamento

Il fornitore sottopone regolarmente alla proponente l'evidenza di quanto realizzato, e dimostra la conformità ai requisiti attraverso *test di sistema*$_G$. La rendicontazione avviene durante incontri programmati ed è meglio guidata dall'esposizione di una demo della funzionalità realizzata.

2.1.6.3 Procedura: validazione del lavoro svolto

Il fornitore e la *committente*$_G$ verificano che il lavoro svolto sia conforme ai requisiti e alle aspettative della proponente, attraverso i test di accettazione.

2.1.6.4 Strumenti

- **Google meet**: per la comunicazione sincrona con la committente.
- **Gmail**: per la comunicazione asincrona con la committente.

2.1.7 Attività: consegna e terminazione

2.1.7.1 Procedura: consegna del prodotto finale

Il fornitore deve assicurarsi che il prodotto sia completo, funzionante e conforme ai requisiti stabiliti, e fornire tutta la documentazione necessaria per l'utilizzo del prodotto. Per fare ciò si utilizza la funzione *releases* di *GitHub*$_G$.

2.1.7.2 Procedura: esposizione delle funzionalità

Il fornitore presenta alla proponente le funzionalità del prodotto, illustrando come soddisfa i requisiti e rispondendo a eventuali domande o dubbi da parte della proponente. Vengono mostrati assieme alla demo i risultati dei *test*$_G$, prova della correttezza di quanto sviluppato.

2.1.7.3 Procedura: esposizione del processo di sviluppo

Il fornitore fornisce una panoramica dettagliata del processo di sviluppo seguito, evidenziando le scelte progettuali, le sfide affrontate e le soluzioni adottate durante lo sviluppo del prodotto, oltre a un'analisi della gestione del progetto concretizzatasi nel Piano di Progetto (Sezione 2.1.8.9). Tale attività coincide con la milestone di consegna *Product Baseline (PB)*$_G$.

2.1.7.4 Strumenti

- **Google meet**: per la comunicazione sincrona con la committente.
- **Gmail**: per la comunicazione asincrona con la committente.
- **GitHub**: attraverso la funzionalità di *release* per la consegna del prodotto finale, garantendo tracciabilità e accessibilità al codice sorgente e alla documentazione associata.

2.1.8 Documentazione fornita

2.1.8.1 Analisi dei capitolati

Il documento di Analisi dei capitolati (v1.0.0) mette in evidenza le considerazioni fatte riguardo ai capitolati presentati. Vengono analizzati complessità, rischi e opportunità formative di ciascun capitolato per determinare la scelta ottimale per il gruppo. **Tipo**: Esterno
Destinatari: Team 7-ZPUs, Prof. Vardanega, Prof. Cardin

2.1.8.2 Lettera di candidatura

Il documento di Lettera di candidatura (v1.0.0) formalizza la candidatura del team ai committenti per lo sviluppo del progetto proposto, dichiarando la data di consegna, il budget stimato e la composizione ufficiale del team.
Tipo: Esterno
Destinatari: Team 7-ZPUs, Prof. Vardanega, Prof. Cardin, Sanmarco Informatica

2.1.8.3 Lettera di presentazione

La lettera di presentazione è il documento che contiene la proposta formale di presentazione del team alle revisioni *RTB*$_G$ e PB dove vengono esposti gli *artefatti*$_G$ presenti della rispettiva *baseline*$_G$.
Tipo: Esterno
Destinatari: Team 7-ZPUs, Prof. Vardanega, Prof. Cardin, Sanmarco Informatica

2.1.8.4 Preventivo costi e assunzione impegni

Documento che definisce l'ammontare orario per ciascun ruolo e per ciascuna persona, con conseguente preventivo di spesa. Il preventivo di spesa viene calcolato in base ai costi orari stabiliti per ciascun ruolo.
Tipo: Esterno
Destinatari: Team 7-ZPUs, Prof. Vardanega, Prof. Cardin, Sanmarco Informatica

2.1.8.5 Verbali interni

I verbali interni documenti che raccolgono le informazioni e le *decisioni*$_G$ prese durante le riunioni di progetto interne al team 7-ZPUs.
Tipo: Interni
Destinatari: Team 7-ZPUs, Prof. Vardanega, Prof. Cardin

2.1.8.6 Verbali esterni

I verbali esterni sono documenti che raccolgono le informazioni e le decisioni prese durante le riunioni di progetto con l'azienda proponente.
Tipo: Esterni
Destinatari: Team 7-ZPUs, Prof. Vardanega, Prof. Cardin, Sanmarco Informatica

2.1.8.7 Analisi dei Requisiti

Nel documento di Analisi dei requisiti (v1.0.0) sono riportati i bisogni e i vincoli a cui attenersi per la realizzazione del prodotto finale. L'obiettivo è definire in maniera non ambigua i casi d'uso (*Use Case*$_G$) e i requisiti (*Requirements*) del software. Il documento è diviso nelle seguenti sezioni:

1. **Introduzione:** Include lo scopo, i riferimenti normativi (come il Capitolato C3) e informativi, e il rimando a un Glossario esterno per la terminologia tecnica.

2. **Descrizione:** Fornisce una panoramica del sistema, le sue funzionalità generali e l'identificazione degli utenti di destinazione.

3. **Definizione dei casi d'uso:** Rappresenta il nucleo del documento, con i casi d'uso principali descritti tramite *diagrammi UML*$_G$ e schede tecniche che includono attori, *precondizioni*$_G$, *postcondizioni*$_G$ e flussi principali.

4. **Definizione dei requisiti:** Classificazione dettagliata dei requisiti individuati

Il tracciamento dei requisiti è automatizzabile tramite *script*$_G$ a partire dai requisiti definiti nel documento di *Analisi dei Requisiti*$_G$.
Tipo: Esterno
Destinatari: Team 7-ZPUs, Prof. Vardanega, Prof. Cardin, Sanmarco Informatica

2.1.8.8 Glossario

Il Glossario è il documento che raccoglie ogni termine di carattere tecnico, nomenclature e acronimi che possono avere interpretazioni divergenti e che possono essere fonte di ambiguità nella comunicazioni tra i vari *stakeholder*$_G$ di progetto. La definizione dei termini di glossario è coadiuvata dal contenuto dello standard ISO/IEC/IEEE 24765/2017.

Il controllo dell'aggiornamento del glossario è automatizzato da un script apposito che controlla la presenza di nuovi termini non ancora inseriti nel documento Glossario. È inoltre disponibile la versione web nella pagina del team, per una maggiore operabilità.

2.1.8.9 Piano di Progetto

Il Piano di Progetto v1.0 (v1.0.0) è il documento che espone all'esterno il lavoro di sviluppo svolto seguendo le procedure delineate all'interno di questo documento. Fornisce una guida dettagliata alla pianificazione, esecuzione e consuntivo delle attività completate in ciascuna *Sprint*$_G$ e funge da guida per l'esecuzione del progetto. Il documento è diviso nelle seguenti sezioni:

1. Introduzione
2. Analisi dei rischi e mitigazione
3. Modello di sviluppo
4. Pianificazione dei costi e suddivisione ruoli
5. Preventivo di periodo
6. Consuntivo di periodo
7. *Retrospettiva*$_G$

Tipo: Esterno
Destinatari: Team 7-ZPUs, Prof. Vardanega, Prof. Cardin, Sanmarco Informatica

2.1.8.10 Piano di Qualifica

Il Piano di Qualifica v2.0 (v2.0.0) è il documento che descrive gli obiettivi di qualità dei processi che il fornitore si impegna a soddisfare per consegnare un prodotto finale di qualità. Le metriche di valutazione vengono determinate dall'analisi dei requisiti e dalle indicazioni date dalla proponente, suddivise in base all'applicazione sui processi o sul prodotto. Le metriche stabilite vengono poi misurate attraverso opportuni test e verifiche, di cui vengono riportate le specifiche. Il documento include una sezione di rendicontazione per la valutazione dei processi e la valutazione del prodotto, in cui riportare l'attinenza alle metriche ottenuta rispetto agli obiettivi e di conseguenza valutare azioni correttive in caso si verifichino eventuali problemi (*cruscotto di qualità*$_G$). Il documento è diviso nelle seguenti sezioni:

1. Qualità dei processi
2. Qualità del prodotto
3. Specifiche di test e verifica

4. Cruscotto di qualità

Tipo: Esterno
Destinatari: Team 7-ZPUs, Prof. Vardanega, Prof. Cardin, Sanmarco Informatica

2.1.8.11 Specifica Tecnica

La Specifica Tecnica v1.0 (v1.0.0) è il documento che descrive in modo rigoroso le scelte tecnologiche e l'architettura del prodotto da sviluppare. Fornisce un'analisi che parte dalla descrizione del contesto applicativo, per poi approfondire la struttura dei container di sviluppo, delle componenti e del codice, al fine di fornire una rappresentazione completa e coerente del sistema.

2.1.8.12 Strumenti

- ***GitHub***$_G$ per la gestione e il *versionamento*$_G$ della documentazione di progetto e mezzo comunicativo nella fase di fornitura
- ***Jira***$_G$ per la suddivisione e il monitoraggio delle attività di progetto
- **Discord** per la comunicazione sincrona tra i membri del gruppo
- **Gmail** per la comunicazione asincrona con l'azienda proponente
- **VSCode** con *estensione*$_G$ *Latex Workshop*$_G$ per permettere di lavorare con facilità ai documenti.

2.1.9 Ruoli

Vengono qui raccolti per praticità i ruoli coinvolti del processo di fornitura, con una breve descrizione delle responsabilità di ciascuno.

Ruolo	Specifica
Responsabile di progetto	Stabilisce, pianifica, coordina e monitora le attività del team. Regola le comunicazioni tra i vari stakeholder e documenta le decisioni in verbali (esterni o interni). Individua e gestisce i rischi che si presentano o potrebbero presentarsi durante lo svolgimento del progetto.
Amministratore	Si occupa del controllo e del mantenimento dell'infrastruttura, assicurando il funzionamento degli strumenti a disposizione del team. Guida l'adozione rigorosa del Way of Working e supporta i membri nell'utilizzo degli strumenti.
Analisti	Figura centrale nella fase di analisi del prodotto; identifica e documenta i requisiti del sistema garantendone chiarezza e completezza. È responsabile della redazione del documento Analisi dei Requisiti (v1.0.0).
Progettista$_G$	Responsabile della progettazione dell'architettura del sistema e delle componenti software, assicurando che queste soddisfino i requisiti definiti nel documento di analisi. Redige il documento di Specifica Tecnica (v1.0.0).
Sviluppatore	Concretizza la progettazione in codice funzionante e realizza i test automatici, seguendo le linee guida stabilite nel Way of Working.
Verificatore	Garantisce che quanto confluisce nella *repository*$_G$ sia corretto; esegue revisioni di codice e documenti, individua lacune o errori e ne segnala la risoluzione.

2.2 Processo di sviluppo

Il processo di sviluppo prevede l'insieme di attività che definiscono lo svolgimento dell'Analisi dei Requisiti, la successiva realizzazione dell'*architettura*$_G$ del sistema, la codifica, i test e la *validazione*$_G$ del prodotto software.

2.2.1 Attività: analisi dei requisiti

L'Analisi dei Requisiti rappresenta una delle attività fondamentali per il raggiungimento della **Requirements and Technology Baseline** (RTB) e ha l'obiettivo di identificare in modo completo l'insieme dei requisiti che il sistema sviluppato dovrà soddisfare.
I risultati di tale attività sono raccolti nel documento *Analisi dei Requisiti*, nel quale sono riportate in maniera dettagliata tutte le informazioni necessarie. Questo documento costituisce un riferimento essenziale per le successive fasi di *progettazione*$_G$ dell'architettura e di codifica, supportando il lavoro dei progettisti e degli sviluppatori.

2.2.1.1 Procedura: raccolta e analisi dei requisiti

Il fornitore deve individuare le esigenze dell'utente finale rispetto alle funzionalità richieste del Software. Un passaggio chiave è pensare il mondo *As is*$_G$ e *To be*$_G$. In questo modo è più facile individuare le funzionalità principali del software. Poi queste vengono approfondite con l'aiuto e la comunicazione costante con la committente. Occorre anche individuare vincoli e requisiti impliciti, derivanti dal dominio in cui opera il software o non considerati dalla committente. Per farlo è utile richiedere riferimenti sul dominio applicativo, come documentazione tecnica, legale o normativa.

2.2.1.2 Procedura: tracciamento dei requisiti

Di fondamentale importanza è il tracciamento dei requisiti-fonti e requisiti-use case. Il primo per garantire di non aver tralasciato nessun aspetto del capitolato o discussione con la committente. Il secondo per garantire che requisiti e casi d'uso siano sempre allineati. Questo per evitare di tralasciare requisiti non modellati in casi d'uso, e che ogni caso d'uso non sia superfluo (non abbia requisiti associati).

2.2.1.3 Procedura: confronto con la committente

Ogni volta che si identificano nuovi requisiti o si chiariscono quelli già individuati, è necessario confrontarsi con la committente per validare le informazioni raccolte e garantire che siano in linea con le aspettative e le esigenze della committente stessa. Il confronto con la committente è un processo iterativo che avviene durante tutto il processo di sviluppo, al fine di garantire che il prodotto finale soddisfi le esigenze e le aspettative della committente.

2.2.1.4 Procedura: produzione del documento di Analisi dei requisiti

Il fornitore redige un documento che raccolga in modo chiaro e dettagliato tutte le informazioni emerse durante l'analisi dei requisiti, organizzando e categorizzando i casi d'uso individuati e i relativi requisiti associati. La produzione avviene come descritto in §3.1.2.
In particolare, il documento di Analisi dei Requisiti organizza i casi d'uso individuati e i relativi requisiti associati. Al fine di agevolarne la consultazione, viene di seguito illustrata nel dettaglio la nomenclatura adottata.

2.2.1.4.1 Casi d'uso

I casi d'uso utilizzano la seguente nomenclatura:

UC-[ID_Radice].[Indici_Annidamento] - [Titolo]

UC sta per Use Case, ovvero caso d'uso in inglese. Gli UC sono identificati univocamente tramite una numerazione crescente.
ID Radice: numero progressivo del caso d'uso, che identifica univocamente il caso d'uso principale.

Indici di annidamento: numerazione progressiva che identifica i casi d'uso secondari, ovvero quelli che rappresentano specifiche dello UC padre.
Esempio: UC-1 - Visualizza dati persona, UC-1.1 - Visualizza Nome, UC-1.2 - Visualizza Cognome...

2.2.1.4.2 Requisiti

I requisiti utilizzano la nomenclatura seguente:

R-[ID]-[Tipo]-[Priorità]

Dove:

- **ID**: numero progressivo del requisito
- **Tipo**:
  - **F** (Requisiti Funzionali): descrivono le funzionalità del *sistema*$_G$
  - **Q** (Requisiti di Qualità): descrivono le caratteristiche qualitative del sistema
  - **V** (Requisiti di Vincolo): descrivono i vincoli tecnologici e normativi
- **Priorità**: **Ob** (Obbligatorio), **De** (Desiderabile), **Op** (Opzionale)

2.2.1.5 Diagrammi dei casi d'uso

Ogni caso d'uso è presentato tramite un diagramma che rispetta la sintassi UML, che rappresenta graficamente quanto scritto. È di fondamentale importanza poiché permette a qualsiasi lettore di comprendere il significato. I diagrammi vengono realizzati secondo il seguente schema:

- **Use case principale**: diagramma contenente il caso d'uso principale, gli attori coinvolti e le sue *estensioni*$_G$
- **Inclusioni**: diagramma contenente le *inclusioni*$_G$ del caso d'uso principale.
- **Generalizzazioni**: diagramma contenente il caso d'uso principale e le sue *generalizzazioni*$_G$.

Non vengono prodotti i diagrammi per casi d'uso secondari, in quanto non apportano valore aggiunto se non maggiore difficoltà di lettura.

2.2.1.6 Strumenti

- **GitHub**: per la gestione della documentazione di progetto e mezzo comunicativo nella fase di fornitura
- **Jira**: per la suddivisione e il monitoraggio delle attività di progetto (*Issue Tracking System (ITS)*$_G$)
- **Discord**: per la comunicazione sincrona tra i membri del gruppo

- **Gmail**: per la comunicazione asincrona con l'azienda proponente
- **VSCode**: con estensione con *IDE*$_G$ di riferimento, utilizzato per la produzione della documentazione.
- **Draw.io**: per la realizzazione dei diagrammi dei casi d'uso.

2.2.2 Attività: individuazione delle tecnologie

Dopo una fase iniziale di esplorazione, il fornitore identifica le tecnologie ritenute adatte allo sviluppo del prodotto. Questa scelta è corroborata da un artefatto software che dimostra la corretta integrazione delle tecnologie (Proof of Concept).

2.2.2.1 Procedura: individuazione delle tecnologie

Il fornitore effettua un approfondito studio delle tecnologie disponibili, valutando le caratteristiche di ciascuna e la loro aderenza ai requisiti del progetto. La scelta delle tecnologie viene poi validata attraverso la realizzazione di un Proof of Concept che dimostri l'efficace integrazione delle tecnologie scelte e la loro capacità di soddisfare i requisiti individuati. Questa procedura non ha una definizione rigida, per questo si lascia libertà nell'eseguirla a seconda delle esigenze dello specifico progetto. E' buona prassi considerare per ogni tecnologia i seguenti aspetti:

- **Supporto**: valutare l'esistenza di documentazione, tutorial, community e risorse di supporto
- **Developer Experience**: valutare la facilità d'uso, la curva di apprendimento in confronto alle alternative
- **Affidabilità**: inteso come stabilità, sicurezza e performance della tecnologia. E' preferibile fare affidamento su tecnologie già stabilite nel mercato, in modo da ridurre il rischio tecnologico
- **Costi**: valutare i costi associati alla tecnologia, come ad esempio costi di licenza.

2.2.2.2 Procedura: produzione del Proof of Concept

Il fornitore sviluppa un Proof of Concept (PoC) per mostrare l'integrazione delle tecnologie scelte e la loro capacità di soddisfare i requisiti individuati. Per questo artefatto si lascia ampio margine di manovra. Non è richiesto lo stesso livello di qualità del prodotto finale. La produzione ha forti limiti di risorse e l'obiettivo è solo dimostrare la fattibilità.

2.2.2.3 Strumenti

- **GitHub**: per la gestione del codice nella fase di fornitura
- **Jira**: per la suddivisione e il monitoraggio delle attività di progetto (*Issue Tracking System (ITS)*$_G$)

- **Discord**: per la comunicazione sincrona tra i membri del gruppo
- **VSCode**: con estensione con *IDE*$_G$ di riferimento, utilizzato per lo sviluppo del PoC (*Proof of Concept*$_G$).

2.2.3 Ruoli

Ruolo	Specifica
Responsabile	Stabilisce, pianifica, coordina e monitora le attività di analisi. Scompone le attività di analisi suddividendole in maniera tale che gli ambiti di analisi delle varie persone sia il più indipendente possibile.
Analista	È responsabile del documento di Analisi dei Requisiti. Il suo compito è identificare e scomporre i requisiti fino a un livello di granularità sufficiente. Si occupa inoltre della produzione dei diagrammi dei casi d'uso.
Verificatore	Verifica che ogni caso d'uso abbia granularità sufficiente, e che i diagrammi rispettino la sintassi UML

2.2.4 Attività: progettazione

L'attività di progettazione definisce l'architettura del sistema e le sue componenti. I requisiti individuati diventano una struttura software chiara e manutenibile. Il gruppo adotta il **modello C4** come riferimento per rappresentare l'architettura su più livelli. I diagrammi possono essere prodotti con **PlantUML** o **Draw.io**. In entrambi i casi, devono rispettare sintassi e convenzioni del modello C4.

2.2.4.1 Procedura: progettazione architetturale (Livelli C4 1 e 2)

I livelli più astratti del modello C4, ovvero il **Context diagram** (livello 1) e il **Container diagram** (livello 2), vengono progettati **collegialmente** durante sessioni sincrone del team. Questo approccio è motivato dalla natura trasversale di tali diagrammi, che coinvolgono scelte architetturali ad alto impatto sull'intero sistema e richiedono l'attenzione di tutti i membri.
Le sessioni si svolgono tramite Discord con condivisione dello schermo, seguendo i seguenti passaggi:

1. **Presentazione del contesto**: la discussione è guidata a partire dai requisiti e dai casi d'uso già definiti nel documento di Analisi dei Requisiti.
2. **Definizione condivisa**: i progettisti propongono e discutono le scelte architetturali, raggiungendo un consenso sulla struttura del sistema e delle sue componenti.

3. **Produzione del diagramma**: uno o più membri redigono il diagramma risultante, rendendolo poi parte della documentazione secondo le modalità descritte nelle norme per la produzione dei diagrammi.

2.2.4.2 Procedura: progettazione di dettaglio (Livelli C4 3 e 4)

I livelli di maggiore dettaglio del modello C4, ovvero il **Component diagram** (livello 3) e il **Code diagram** (livello 4), vengono progettati **individualmente** dai membri assegnati alle rispettive componenti. Questo approccio consente una maggiore *efficienza*$_G$, distribuendo il carico di lavoro tra i membri del team e lasciando a ciascuno la responsabilità della componente assegnata.
Il processo di produzione individuale prevede i seguenti passaggi:

1. **Ricezione del contesto**: il progettista assegnato prende visione dei diagrammi di livello superiore già approvati, assicurandosi di rispettare i vincoli architetturali stabiliti collegialmente.

2. **Produzione del diagramma**: il progettista redige il diagramma della componente assegnata, applicando le norme di nomenclatura definite e le modalità descritte nelle norme per la produzione dei diagrammi.

3. **Verifica**: il diagramma prodotto viene sottoposto a verifica da parte di un altro membro del team secondo le procedure definite in 3.2, al fine di garantire coerenza con l'architettura complessiva e correttezza formale.

2.2.4.3 Norme per la produzione dei diagrammi

Indipendentemente dal livello C4 e dallo strumento utilizzato, i diagrammi devono rispettare le seguenti linee guida:

- I nomi degli elementi (componenti, container, relazioni) devono essere **significativi**, chiari e coerenti con la terminologia del dominio applicativo e del Glossario.

- I diagrammi di livello 1 e 2 devono essere **validati dal team** prima di essere integrati; quelli di livello 3 e 4 devono essere **verificati** da almeno un membro diverso dall'autore.

- I diagrammi prodotti con **PlantUML** vengono versionati direttamente nella repository GitHub di progetto come file testuali `.puml`.

- I diagrammi prodotti con **Draw.io** vengono caricati temporaneamente su Google Drive per la condivisione e la revisione; una volta approvati, vengono esportati come immagine e inseriti nella documentazione.

2.2.4.4 Strumenti

- **PlantUML**: strumento testuale per la produzione dei diagrammi C4, integrato in VSCode tramite apposita estensione.
- **Draw.io**: strumento grafico alternativo per la produzione dei diagrammi C4, utilizzabile quando si preferisce un approccio visuale.
- **Discord**: per le sessioni collegiali di progettazione architetturale (livelli C4 1 e 2), con condivisione dello schermo per la produzione condivisa dei diagrammi oppure per chiarire dubbi.
- **GitHub**: per il versionamento dei file PlantUML e la gestione delle Pull Request di verifica dei diagrammi di dettaglio (livelli C4 3 e 4).
- **Google Drive**: per la condivisione temporanea dei diagrammi Draw.io in fase di revisione.
- **VSCode**: IDE principale, con estensione PlantUML per la visualizzazione in *anteprima*$_G$ dei diagrammi durante la stesura.
- **Jira**: per la creazione e il tracciamento delle task di progettazione, seguendo le stesse procedure definite per le altre attività in 4.1.1.1.

2.2.5 Attività: codifica

L'attività di codifica ha l'obiettivo di produrre unità software corrette, leggibili e manutenibili, in coerenza con i requisiti approvati e con l'architettura definita dal team.

2.2.5.1 Procedura: definizione e applicazione delle norme di codifica

Prima dell'implementazione, il gruppo definisce norme condivise di stile e struttura del codice, aggiornandole quando necessario durante il progetto. L'adozione di regole comuni riduce ambiguità e facilita la revisione.
In particolare, la codifica deve rispettare le seguenti linee guida generali:

- Separare in modo chiaro responsabilità di ***front-end***$_G$, ***back-end***$_G$ e servizi di integrazione, limitando l'accoppiamento tra componenti.
- Mantenere le interfacce tra moduli esplicite e stabili, evitando dipendenze non necessarie.
- Applicare uno stile uniforme di formattazione e nomenclatura all'interno del repository.

nello specifico, vengono definite norme di nomenclatura per variabili, funzioni, classi e file, che devono essere seguite da tutti i membri del team.

- Utilizzo di `camelCase` per variabili e funzioni.

- Utilizzo di `PascalCase` per classi ed enum.
- Utilizzo di `UPPER_SNAKE_CASE` per costanti ed elementi di un enum.
- Ogni interfaccia deve essere preceduta dal prefisso `I` (es. `IUserService`). La rispettiva implementazione deve avere lo stesso nome senza prefisso (es. `UserService`).
- Ogni file deve contenere la definizione di al più una classe, nonchè possedere lo stesso nome della classe principale. Fanno eccezione i casi di classi utilizzate esclusivamente come supporto alla classe principale (es. classi di test, classi di eccezione, classi di utilità). In questo caso, il file deve essere nominato comunque con il nome della classe principale (es. `UserService.ts` può contenere la classe `UserService` e altre classi di supporto).
- Qualsiasi nome deve essere significativo e descrittivo, anche a discapito di nomi più brevi. Ad esempio, è preferibile utilizzare `getUserById` invece di `getUser`, in quanto il primo è più esplicativo riguardo alla funzionalità della funzione.
- Per le classi del main process, è consentita esclusivamente l'utilizzo di Dependency Injection tramite constructor injection con appositi decoratori. Non è consentita la risoluzione esplicita con chiamate al container DI.

2.2.5.2 Procedura: implementazione delle unità software

Ogni modifica deve essere realizzata in unità coerenti e verificabili, privilegiando semplicità e chiarezza. Durante la codifica, il team adotta le seguenti buone pratiche:

- evitare l'uso di variabili globali o pattern *singleton*$_G$ non adeguatamente giustificati;
- mantenere funzioni e unità software con una sola responsabilità, evitando implementazioni eccessivamente lunghe;
- utilizzare nomi significativi per variabili, funzioni, classi e file;
- inserire commenti solo dove utili a chiarire decisioni non ovvie, evitando ridondanza con il codice.

2.2.5.3 Strumenti

- **IDE**: per la scrittura del codice e il supporto all'analisi statica in tempo reale.
- **Linter e formatter**: per applicare automaticamente le convenzioni di stile e ridurre difetti formali.
- **Sistema di versionamento**: per tracciare le modifiche, gestire *branch*$_G$ e revisioni.

Inoltre, al fine di allineare tutti i membri del team sulle stesse tecnologie e strumenti, è necessario l'utilizzo dell'ambiente di sviluppo condiviso a container Docker.
Così facendo si limitano i problemi di configurazione e compatibilità.

2.2.6 Attività: realizzazione dei test

L'attività di test ha l'obiettivo di verificare in modo sistematico la correttezza del software prodotto e la conformità ai requisiti, riducendo il rischio di regressioni.

2.2.6.1 Procedura: Pianificazione dei test

Per ogni incremento vengono identificati i casi di test necessari, definendo copertura attesa e priorità in funzione del rischio e della criticità delle funzionalità coinvolte. Vengono definite a seguire le linee guida da seguire per la definizione dei test:

- I test devono essere progettati per essere **indipendenti** l'uno dall'altro, in modo da poter essere eseguiti in qualsiasi ordine senza influenzare i risultati.
- I test devono essere **ripetibili**, restituendo gli stessi risultati se eseguiti più volte con le stesse condizioni.
- I test devono essere **automatizzati** quando possibile, integrandoli nel processo di sviluppo per garantire feedback rapido e continuo.
- I test devono essere **documentati** in modo chiaro, specificando lo scopo del test, i dati di input, i risultati attesi e le condizioni di esecuzione. Questo avviene direttamente nel codice tramite **commenti** che decorano la definizione del test.

2.2.6.2 Procedura: Implementazione ed esecuzione dei test

I test vengono realizzati a livelli progressivi, con esecuzione automatica durante lo sviluppo, integrata nel workflow. Quando possibile, i test sono automatizzati per garantire ripetibilità e rapidità di feedback. I livelli di test adottati sono i seguenti:

- **Unit Testing**: test di unità per verificare la correttezza di singole unità di codice.
- **Integration Testing**: test di integrazione per verificare l'interazione tra diverse unità o componenti del sistema.
- **System Testing**: test di sistema per verificare il comportamento complessivo del sistema rispetto ai requisiti specificati.

Maggior dettaglio sui livelli di test e le relative procedure di implementazione ed esecuzione è riportato nella sezione 3.3.4.

2.2.6.3 Procedura: analisi degli esiti e azioni correttive

Gli esiti dei test vengono analizzati dal team per identificare difetti, aprirne tracciamento e pianificare la correzione. Ogni correzione deve essere accompagnata da verifica di non regressione.

2.2.6.4 Strumenti

- **Framework di test**: per la scrittura ed esecuzione dei test automatizzati. In particolare:
  - **Vitest**: Strumento di testing per i test di unità dei singoli componenti. È stato sfruttato anche per i test di integrazione dei diversi moduli.
  - **Playwright**: Strumento di testing per i test end-to-end, che permettono di testare il sistema nella sua interezza, simulando l'interazione dell'utente con l'interfaccia grafica.
- **Pipeline di integrazione continua**: per l'esecuzione ricorrente dei test e il controllo qualità.
- **Strumenti di tracciamento**: per registrare difetti, priorità ed esito delle attività correttive.

2.2.7 Ruoli

Vengono qui raccolti per praticità i ruoli coinvolti dell'attività di progettazione, con una breve descrizione delle responsabilità di ciascuno.

Ruolo	Specifica
Responsabile	Coordina le sessioni collegiali di progettazione architetturale, assegna le componenti per la progettazione di dettaglio e monitora la coerenza complessiva dell'architettura.
Progettista	Figura centrale dell'attività: partecipa attivamente alle sessioni collegiali per i livelli C4 1 e 2, e produce autonomamente i diagrammi di dettaglio dei livelli C4 3 e 4 per le componenti assegnate.
Verificatore	Verifica la correttezza e la coerenza architetturale dei diagrammi di dettaglio (livelli C4 3 e 4) prodotti individualmente, secondo le procedure di revisione definite in 3.2.

3 Processi di supporto

I processi di supporto sono complementari ai processi primari. Questi processi non producono il software in sé, ma forniscono gli strumenti e le verifiche per il corretto completamento di quelli primari. La relazione che intercorre tra le due tipologie di processi è descrivibile come "dipendenza funzionale". Un processo primario (es. Sviluppo) non può terminare con successo senza l'intervento costante dei processi di supporto. I processi di supporto d'altra parte non hanno senso se non relazionati a dei processi primari.

3.1 Processo di documentazione

Il processo di documentazione è parte integrante del Progetto in quanto permette il tracciamento delle decisioni prese, delle attività svolte e dei risultati ottenuti. Tutto ciò al fine di favorire il lavoro asincrono tra membri del gruppo. La documentazione rappresenta un elemento imprescindibile del processo di progetto: trasforma conoscenze implicite e decisioni verbali in artefatti formali, garantendo coerenza interpretativa e tracciabilità. Documentare permette di ridurre le ambiguità e rendere riproducibili le scelte adottate, facilitando verifiche e future modifiche. Per essere efficace, la documentazione deve essere tempestiva, chiara e reperibile.

3.1.1 Attività: pianificazione della documentazione

La pianificazione della documentazione avviene insieme alla pianificazione delle attività di progetto.
Durante ogni Sprint, il responsabile di progetto assegna le attività ai membri del gruppo. Tiene conto delle competenze e della disponibilità di ciascuno. Le scadenze vengono fissate per mantenere la documentazione aggiornata e disponibile anche per eventuali attori esterni (azienda proponente e committente).
Per scrivere più facilmente i documenti periodici (Verbali Interni, Verbali Esterni, Diario di Bordo), sono disponibili modelli standard approvati in /assets. L'aggiornamento di tali standard deve essere argomento di Verbali Interni e risultato di una discussione e successiva decisione presa in tale sede.

3.1.1.1 Procedura: identificazione della documentazione

Durante la pianificazione di ogni Sprint, il responsabile identifica le necessità di documentazione in base agli obiettivi dello Sprint e alle attività previste. Ad ogni documento principale è legata una branch denominata `x_in_lavorazione` che mantiene le versioni aggiornate pronte alla verifica finale di periodo.

3.1.1.2 Procedura: creazione delle attività

Il responsabile assegna le attività di documentazione ai membri del gruppo, tenendo conto delle competenze e della disponibilità di ciascuno, nonché della necessità di ruotare i ruoli, per dare la possibilità a tutti i membri di acquisire esperienza in diverse aree.
Vengono create le *issue*$_G$ in Jira per ogni attività di documentazione, specificando i dettagli del compito, le scadenze e i verificatori per ogni attività. Specifiche in 4.1.1.1.

3.1.1.3 Strumenti

- **Jira**: per la gestione delle attività di progetto. In particolare con la *board Scrum*$_G$ che viene aggiornata in automatico con i *commit*$_G$ effettuati sui Work Item e può essere personalizzata con la creazione di Sprint.

- ***Dashboard***$_G$ di Jira per il monitoraggio delle attività assegnate per ogni membro del gruppo.

3.1.2 Attività: produzione della documentazione

La produzione della documentazione è un'attività fondamentale per garantire la tracciabilità e la comunicazione efficace all'interno del gruppo e con gli stakeholder esterni. Ogni documento deve essere redatto seguendo le linee guida stabilite, utilizzando i modelli approvati e rispettando le scadenze stabilite durante la pianificazione.

3.1.2.1 Linee guida per la produzione della documentazione

Per garantire la coerenza e la qualità della documentazione, il gruppo segue le seguenti linee guida:

- **Verbali:** la stesura dei verbali utilizza un template standard approvato per garantire continuità e chiarezza nella comunicazione. La struttura prevede nel seguente ordine:
  - Frontespizio con logo del gruppo, data, durata della riunione e luogo (fisico o virtuale);
  - Indice dei contenuti;
  - Elenco dei partecipanti;
  - Ordine del giorno;
  - Resoconto dettagliato dei punti discussi;
  - Tabella di definizione dei ruoli (se necessario);
  - Sezione dedicata alla retrospettiva (solo verbali interni di fine Sprint)
  - Decisioni prese e tabella delle attività associate. (Solo verbali interni)

  Le decisioni prese nei verbali esterni vengono solamente menzionate, sviluppate successivamente nei verbali interni e riportate nella sezione apposita con tracciamento alle issue Jira.

- **Diario di bordo:** la struttura dei diari di bordo segue anch'essa un template standard che include il logo del gruppo, la data, un numero progressivo, e il numero del gruppo. Le sezioni principali sono:
  - Frontespizio
  - Difficoltà affrontate;
  - Dubbi e incertezze;
  - Pagina conclusiva.

- **Altri documenti:** per tutti gli altri documenti non vengono utilizzati modelli particolari in quanto prevediamo una sola istanza di ciascuno di essi. Tuttavia seguiamo delle linee guida comuni per garantire consistenza dell'impianto tipografico. In particolare:
  - Frontespizio con logo e titolo del documento;
  - Tabella di versionamento;
  - Indice dei contenuti.

3.1.2.1.1 Denominazione e datazione documentazione

Per una corretta archiviazione e reperibilità delle modifiche apportate ai documenti, è necessario seguire le seguenti convenzioni per la datazione e denominazione.

- Tutti i file dei documenti devono seguire la regola del **Pascal Case**, ovvero devono essere scritti senza spazi e con la prima lettera di ogni parola in maiuscola.
- All'interno dei documenti, la data deve essere riportata nel *formato*$_G$ **YYYY-MM-DD**, anno-mese-giorno.

La denominazione dei file sul sito segue anch'essa la datazione usata per il nome dei file su GitHub ovvero **YYYY-MM-DD**, con l'aggiunta del numero di versione alla fine del nome. Per le specifiche di versionamento, si rimanda alla sezione 3.2.3.1.

3.1.2.2 Procedura: revisione e approvazione

Una volta ricevuta la notifica della PR da revisionare, il responsabile dovrà controllare l'aderenza ai modelli approvati, la correttezza formale e sostanziale del documento. Per velocizzare, oltre alla lettura attenta, si consiglia l'uso di LLM, in particolare per l'analisi grammaticale e stilistica.
Completata la revisione, procede come descritto in 3.2.2 con l'approvazione o la richiesta di modifiche.
Lo stesso effettua il *merge*$_G$ da "*in_lavorazione*" nel ramo principale *main* come definito in 3.2.2, base per la versione ufficiale di rilascio corrispondente alla milestone. Questo passaggio dovrebbe risultare puramente formale se la documentazione è stata gestita fino a quel momento rispettando le norme definite. Non di meno il responsabile è il garante delle qualità del documento quindi deve impiegare il proprio tempo, il minimo possibile, per rileggere e confermare i contenuti.

3.1.2.3 Strumenti

- ***VSCode***$_G$ come IDE principale per la stesura dei documenti in LaTeX.
- **Estensione Jira per VSCode** per la gestione dei Work Item assegnati e la creazione automatica delle branch di lavoro.

- **GitHub**, integrato in VSCode per la gestione delle versioni e delle *Pull Request (PR)*$_G$, comodamente nell'ambiente di sviluppo 4.2.1.2. Per una stesura efficiente dei documenti il Team si è dotato di modelli predefiniti (Decisione del 2025-11-07).
- **Jira**: strumento di gestione delle attività di progetto, utilizzato per tracciare le attività di documentazione e assegnarle ai membri del gruppo.
- ***LLM***$_G$ per il supporto alla revisione formale e stilistica dei documenti.

3.1.3 Ruoli

Vengono qui raccolti per praticità i ruoli coinvolti del processo di documentazione, con una breve descrizione delle responsabilità di ciascuno.

Ruolo	Specifica
Responsabile di progetto	Stabilisce, pianifica, coordina e monitora le attività del team. È anche il membro a cui sono assegnate le attività di scrittura di preventivo e consultivo.
Amministratore	Si occupa del controllo e del mantenimento dell'infrastruttura, assicurando il funzionamento degli strumenti come VSCode e GitHub per la documentazione.
Analisti	Figura centrale nella fase di analisi del prodotto; identifica e documenta i requisiti del sistema garantendone chiarezza e completezza. È responsabile della redazione del documento Analisi dei Requisiti (v1.0.0).
Verificatore	Garante della qualità della documentazione, verifica i contenuti, la forma e la struttura dei documenti secondo le norme comuni e specifiche del progetto.

3.2 Processo di Gestione della Configurazione

Il processo di gestione della configurazione ha lo scopo di identificare, definire e controllare gli artefatti che compongono la baseline di progetto (*CI*$_G$) durante tutto il *ciclo di vita*$_G$ del prodotto, garantendo la tracciabilità delle modifiche e l'*integrità*$_G$ dei rilasci.

3.2.1 Attività: identificazione della configurazione

Questa attività prevede la definizione degli elementi della configurazione (CIs) e la loro identificazione univoca.

3.2.1.1 Procedura: identificazione dei CIs

Si identificano come CIs gli artefatti che compongono una baseline di progetto. Nel caso di documenti ricorrenti come i verbali, si raggruppano più documenti omogenei in un unico CI (e.g. "Verbali"). Nel caso di codice si identificano le componenti principali,

non ogni singola unità. Questa scelta lascia al team una gestione più flessibile. I CIs sono sempre associati a un *Epic*$_G$ di Jira. In questo modo si garantiscono tracciabilità e gestione delle modifiche. In particolare, il processo di creazione deve seguire i seguenti passaggi:

1. Creazione di una issue in Jira per ogni nuovo artefatto da produrre (documento, componente software, etc).
2. Identificazione della versione di correzione corrispondente alla milestone RTB o PB.
3. Aggiunta di label specifici per facilitare la ricerca, la categorizzazione dei CIs e la raccolta dei dati per le metriche di qualità. Per esempio: , per esempio *DOCS*, *Formazione*, *Code* etc.

3.2.1.2 Strumenti

- **Jira:** Per il tracciamento dei CIs e delle modifiche apportate ad essi.

3.2.2 Attività: controllo della configurazione

Il controllo della configurazione è il processo con il quale vengono gestite le richieste di modifica ai CIs. Ogni modifica è regolamentata da norme precise per garantire ordine e tracciabilità. Come apportare le modifiche è descritto a seguire.

3.2.2.1 Procedura: creazione delle modifiche

Ogni modifica a un CI deve provenire da un branch dedicato, collegato ad un work item Jira corrispondente, garantendo tracciabilità e controllo delle modifiche. I seguenti passaggi guidano il Team nella produzione di modifiche alla baseline:

1. Consultando l'estensione "Atlassian: Jira, Rovo Dev, Bitbucket" (ultimo accesso: 12/12/2026) il membro del gruppo potrà avere accesso al Work Item assegnatogli e cliccando su "Start Work" potrà **creare il branch** automaticamente nominato secondo quanto specificato in 3.2.2.4. Nell'immagine sottostante sono indicati visivamente i passi.

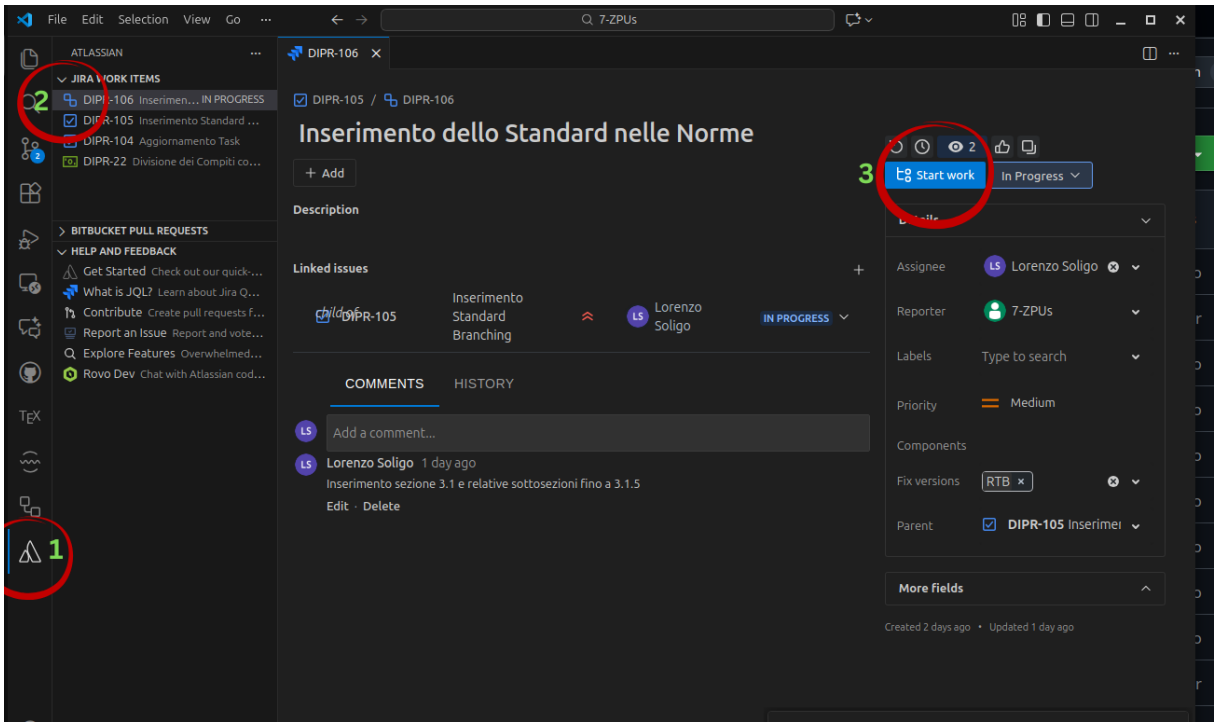

Figura 1: Creazione del branch di lavoro tramite estensione Jira in VSCode

2. Quando è necessario **effettuare un commit** è obbligatorio utilizzare lo Smart Commit. Più in 4.2.1.1.

3. Una volta completata la task, è necessario **creare la PR** verso la branch *"in_lavorazione"* di riferimento mettendo come revisore il membro del gruppo designato.
   Per esempio, una volta completata la stesura di un verbale viene creata una PR verso *verbali_in_lavorazione* e se questa viene approvata, il documento sarà pronto per la revisione finale del responsabile che si occuperà del merge con il *main* in sede di milestone.

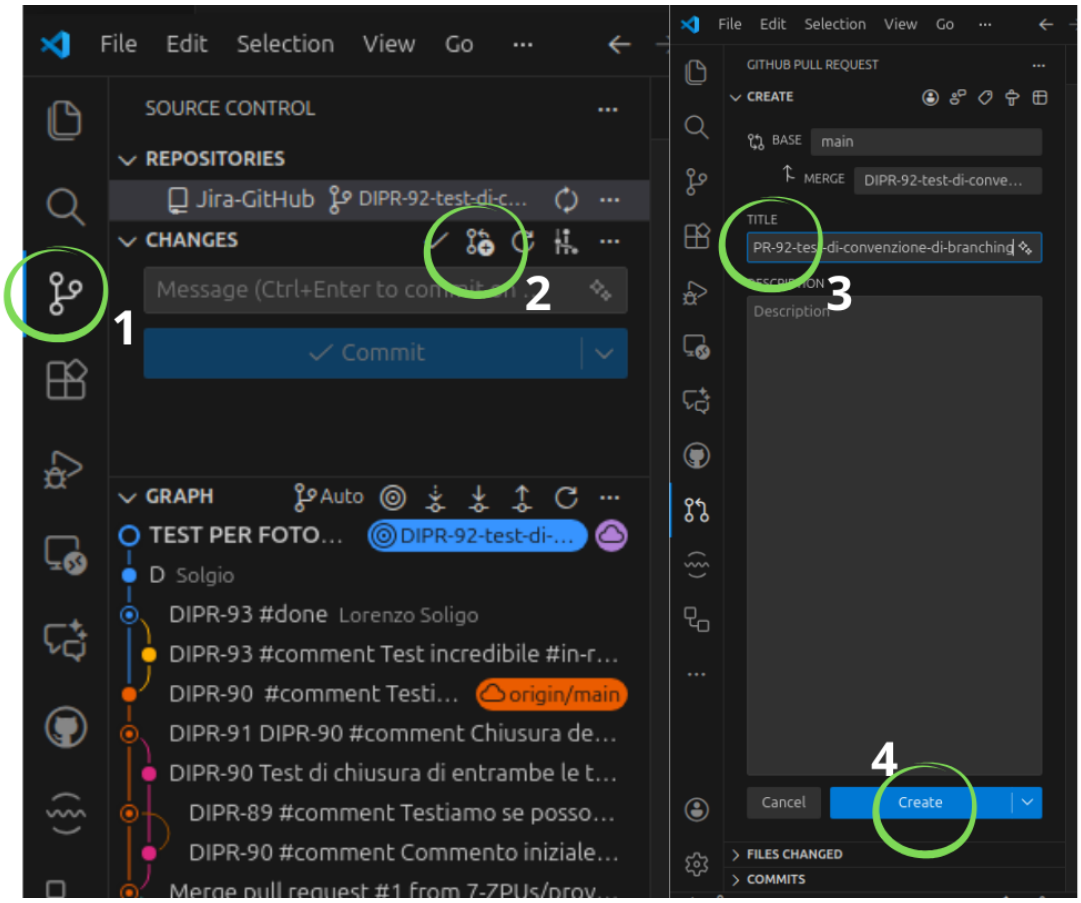

Figura 2: Creazione della PR verso l'issue branch *in_lavorazione*

3.2.2.2 Procedura: controllo della configurazione

Una volta effettuata una modifica nel branch relativo alla task, l'autore crea una Pull Request (PR) dal branch associato alla task verso il branch di riferimento *x_in_lavorazione* (dove x è il nome del CI), impostando come revisore il membro del gruppo designato alla verifica di tale task. Il revisore ha il compito di controllare che la modifica sia corretta e conforme alle norme definite, e di approvare o rifiutare la PR in base alla sua valutazione. A questo punto si verificano i seguenti scenari:

- **Approvazione**: Il revisore procede con l'approvazione delle modifiche e la chiusura della PR tramite l'opzione "Create Merge Commit". Inoltre, specifica nello smart commit la direttiva "#done" per chiudere automaticamente la relativa task su Jira.

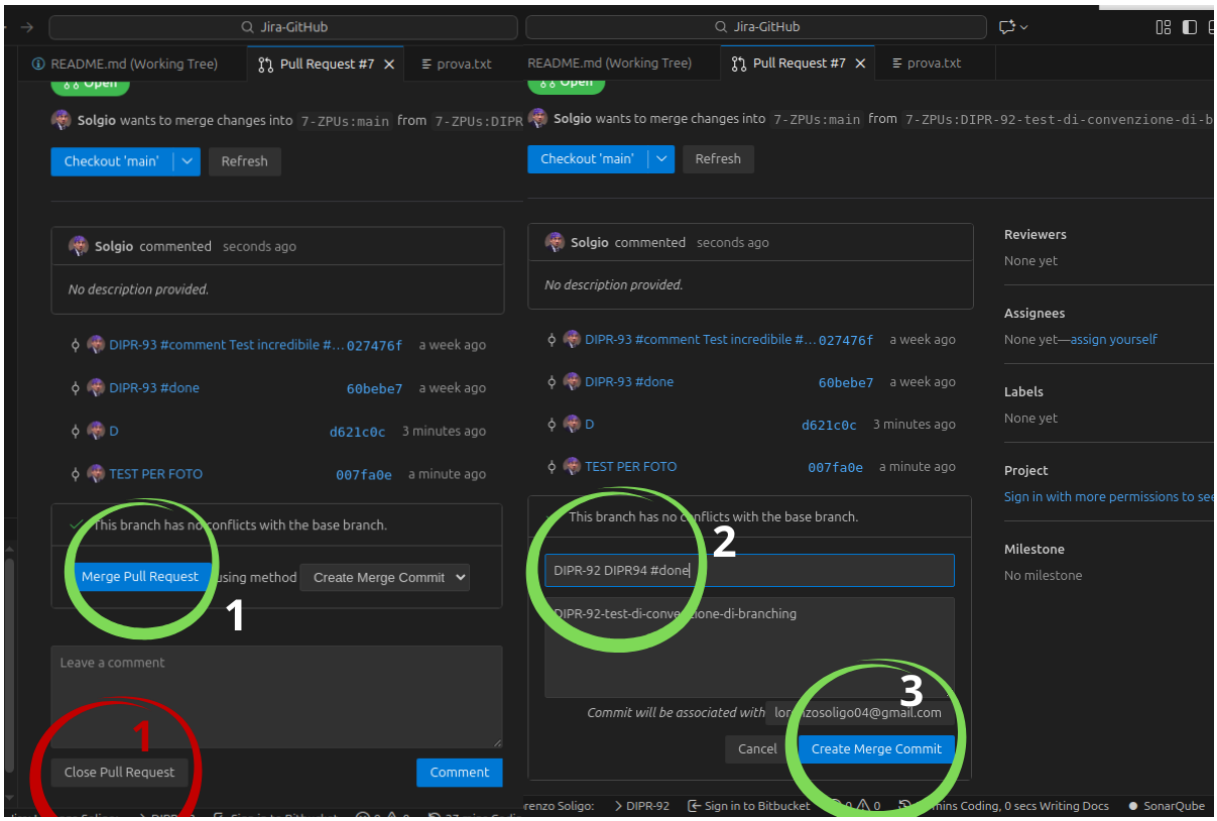

Figura 3: Chiusura della PR tramite "Create Merge Commit"

- **Richiesta di modifiche**: Il revisore fornisce un feedback puntuale all'autore tramite la funzione di review di GitHub. Una volta inseriti i commenti necessari all'autore, il revisore completa la review con l'esito: *"Request changes"*. L'autore provvede a correggere i punti segnalati o a discuterne eventualmente tramite l'apposita sezione. Il revisore provvede ad una nuova revisione dei cambiamenti. Questo ciclo si ripete fino a quando i cambiamenti non vengono approvati.

- **Rifiuto**: Se la task viene annullata il revisore completa la review con l'esito: *"Close with comment"*, inserendo un commento che spiega le motivazioni del rifiuto.

3.2.2.3 Smart Commit

La necessità di tracciamento delle attività richiede l'adozione capillare degli Smart Commit per collegare i commit GitHub alle issue Jira. Questo permette di aggiornare automaticamente lo stato delle task e rendicontare il tempo.
La sintassi obbligatoria è:

```
DIPR-<numero issue> #time <nd nh nm> #<stato> #comment <Descrizione>
```

Regole di applicazione:

1. **Time Tracking:** L'inserimento del tempo (`#time`) deve seguire il formato *nd nh nm* (giorni, ore, minuti).

2. **Transizioni di Stato:** Utilizzare i tag `#start-progress`, `#start-review`, `#complete` per avanzare il *workflow*$_G$ su Jira.
3. **Pull Request:** Nelle PR è vietato inserire il tag `#time` per evitare la doppia contabilizzazione delle ore lavorative.
4. **Issue Multiple:** È possibile agire su più issue in un singolo commit separandole con spazi (utile per chiudere issue di sviluppo e verifica contemporaneamente).

3.2.2.4 Standard per le Branch

Per garantire una gestione ordinata e coerente del codice e della documentazione, il Team adotta il seguente standard per la denominazione dei branch in GitHub:

- La creazione del branch deve avvenire preferibilmente in modo automatico tramite l'integrazione Jira-VSCode (vedi §4.2.2.2.4).
- Il formato obbligatorio è:

```
DIPR-<numero issue>-<descrizione-breve>
```

- È necessario scegliere una descrizione breve che identifichi chiaramente il contenuto della modifica.

3.2.2.5 Strumenti

- **Git/GitHub:** per la gestione di PR e branch.
- **Jira:** per definire e tracciare i CIs e le relative task di modifica.

3.2.3 Attività: registrazione delle configurazioni

Questa attività definisce le regole per la registrazione dei cambiamenti e il versionamento dei CIs.
Nota: il concetto di versione di un documento non corrisponde a quello di versione su GitHub. Per versione in questa sezione si intende una versione stabilita come conseguenza di modifiche apportate all'artefatto di riferimento. Concetto che non coincide con quello di versione all'interno di GitHub.

3.2.3.1 Procedura: registrazione delle Modifiche

Per ogni modifica apportata a uno CI, è necessario documentare le seguenti informazioni, all'interno della sezione denominata "Tabella di Versionamento", situata all'inizio di ogni documento.
Al suo interno vi sono:

- Numero di versione

- Data della modifica (di più in 3.1.2.1.1)
- Autore della modifica
- Verificatore della modifica
- Descrizione della modifica, con riferimento preciso, al paragrafo o sezione interessata.

Per la numerazione delle versioni si adotta lo standard che segue:

- **MAJOR** viene incrementato per modifiche sostanziali che introducono cambiamenti significativi o incompatibili con le versioni precedenti.
- **MINOR** viene incrementato per l'aggiunta di nuove funzionalità o miglioramenti che non compromettono la compatibilità con le versioni precedenti.
- **PATCH** viene incrementato per correzioni di bug, miglioramenti minori o modifiche che non influenzano le funzionalità principali del documento.

Ogni volta che viene apportata una modifica ad uno CI, è necessario aggiornare la tabella di versionamento con le informazioni sopra descritte. Lo scatto di versione avviene solamente in seguito all'approvazione delle modifiche da parte del verificatore stabilito.

3.2.3.2 Strumenti

- **GitHub:** per tracciare le modifiche e collegarle alle issue Jira.
- **Jira:** per tracciare le task che portano alla modifica degli artefatti della baseline.

3.2.4 Ruoli

Vengono qui raccolti per praticità i ruoli coinvolti del processo di gestione della configurazione, con una breve descrizione delle responsabilità di ciascuno.

Ruolo	Specifica
Responsabile di progetto	Figura coordinativa del gruppo che si occupa della identificazione, assegnazione e controllo di avanzamento dei task di modifica.
Amministratore	Si occupa del controllo dell'utilizzo corretto degli strumenti per la gestione delle configurazione. È il referente per l'uso corretto degli strumenti.
Verificatore	Responsabile del controllo in seguito alle modifiche. È suo il compito di verificare che le modifiche apportate siano conformi alle norme e agli standard del progetto.
Esecutore dell'attività	Esegue le attività di modifica seguendo le convenzioni definite come l'uso di Smart Commit e delle Pull Request.

3.3 Processo di verifica

Il processo di verifica è oggettivo e tecnico; ha lo scopo di accertare che i prodotti realizzati (documenti e codice) siano conformi ai requisiti specificati e corretti nella loro implementazione. La verifica viene svolta non solo su elementi software ma anche documentali. A tal proposito, la parte di verifica riguardante gli artefatti software verrà affrontata successivamente al raggiungimento della milestone RTB. E' importante sottolineare che il ruolo della verifica differisce da quello della validazione. La verifica si concentra sulla correttezza di quanto prodotto, mentre la validazione si focalizza sull'adeguatezza del prodotto rispetto alle esigenze del cliente. La verifica è un processo interno al team, mentre la validazione coinvolge anche la committente.

3.3.1 Analisi statica

L'analisi statica consiste nell'esaminare codice e documentazione senza eseguire il software, al fine di individuare errori o non conformità alle norme. L'analisi statica viene effettuata mediante:

- **Walkthrough**: analisi esplorativa del codice o del testo senza conoscenza preventiva della posizione dell'errore. I verificatori individuano le anomalie, mentre la correzione è a carico degli autori.
- **Inspection**: analisi mirata di specifiche porzioni di codice o testo in cui si sospetta la presenza di errori, basata sull'esperienza e su conoscenze pregresse. Assai più efficiente ed automatizzabile rispetto al walkthrough, ma richiede esperienza, spesso a valle di più walkthrough.

3.3.2 Analisi dinamica

L'analisi dinamica consiste nell'eseguire il software per verificarne il corretto comportamento. I test devono essere ripetibili e automatizzabili, isolati e il più efficienti possibili per garantire efficienza. Devono essere inoltre consistenti, ovvero devono restituire sempre lo stesso risultato a parità di condizioni.
I test sono definibili tramite driver, ovvero un componente che simula il comportamento dell'entità "chiamante" verso le unità "chiamate". Lo stub invece costituisce un componente che simula il comportamento dell'entità "chiamata".

3.3.3 Attività: verifica documentale

La verifica documentale consiste nel controllo dei documenti prodotti prima della loro approvazione. La verifica avviene su più fronti:

- **Contenuto**: correttezza dei contenuti, che devono soddisfare le aspettative definite per la task di riferimento
- **Correttezza grammaticale e leggibilità**: i documenti devono essere scritti in modo chiaro, con una grammatica corretta e un livello di leggibilità adeguato al pubblico di riferimento.

- **Aderenza alle linee guida**: i documenti devono rispettare le linee guida stabilite per la produzione della documentazione, come descritto in 3.1.2.1.

3.3.3.1 Procedura: verifica del contenuto documentale

La verifica del contenuto documentale avviene tramite **walkthrough**, con eventuale ausilio di LLM. Il controllo del contenuto avviene corrispondentemente alle aspettative definite per la singola task. È buona prassi assegnare come verificatore un membro che si ritiene più competente in merito al contenuto del documento, in modo da garantire una verifica più accurata e puntuale. Ogni documento deve rispettare quanto descritto in 3.1.2.1.

3.3.3.2 Procedura: verifica di correttezza grammaticale e di leggibilità

La verifica di correttezza grammaticale e di leggibilità avviene tramite script automatizzati. Lo script è contenuto nella directory `scripts`. Oltre al controllo locale, una volta aperta la Pull Request per candidare le modifiche, vengono eseguiti dei workflow automatizzati che prevedono un controllo di grammatica e leggibilità tramite Aspell e Textstat.

3.3.3.3 Procedura: verifica di aderenza alle linee guida

La verifica di aderenza è attuata in forma di checklist. Il verificatore, durante la revisione del documento, si assicura che sia conforme a quanto specificato in 3.1.2.1

3.3.3.4 Strumenti

- **LLM** per il supporto alla verifica del contenuto, in particolare per l'analisi di coerenza, completezza e aderenza alle linee guida.
- **Aspell** per il controllo ortografico, utilizzato negli script automatizzati (*Github Action*$_G$).
- **Textstat** per il controllo di leggibilità, utilizzato negli script automatizzati (Github Action).

3.3.4 Attività: verifica del codice

La verifica statica del codice avviene con SonarQube, che permette di individuare errori, vulnerabilità e code smell nel codice sorgente. Il processo è integrato nel workflow di sviluppo. I controlli partono in automatico a ogni push o pull request.
Inoltre, il Team può consultare la dashboard di SonarQube per monitorare la qualità del codice a livello di branch e di progetto.
La verifica dinamica del codice, come già anticipato nella sezione 2.2.6.2, avviene in tre livelli, descritti in seguito.

3.3.4.1 Procedura: test di unità

Ogni componente software deve essere accompagnato da un test di unità che ne verifichi il comportamento in isolamento. Visto la natura del test, eventuali dipendenze vengono sostituite da *mock*$_G$.
Utilizzando i comandi Angular, nella sezione frontend, è possibile generare automaticamente i file di test da popolare. Si consiglia quindi di sfruttare al massimo le potenzialità di Angular. Per avviare i test di unità è sufficiente eseguire il comando `npm run test` nella root del progetto, che avvierà l'esecuzione dei test frontend e backend.
I test di unità possono essere di due tipi:

- **Test funzionali**: verificano il codice secondo una logica "black-box", ovvero senza conoscenza della struttura implementativa interna. Solitamente utilizzati per verificare che l'unità rispetti la specifica funzionale. Per realizzare test esaustivi è necessario definirli secondo le "classi di equivalenza":
  - **Valori ammissibili**: valori che rispettano le condizioni di validità specificate per l'unità.
  - **Valori non ammissibili**: valori che violano le condizioni di validità specificate per l'unità.
  - **Valori limite**: valori che si trovano ai confini delle condizioni di validità specificate per l'unità.
- **Test strutturali**: verificano il codice secondo una logica "white-box", ovvero con conoscenza della struttura implementativa interna, sono detti anche "white box" tests.

3.3.4.2 Procedura: test di integrazione

I test di integrazione verificano l'interazione tra più componenti, assicurando che funzionino correttamente insieme. È utile effettuare test di integrazione graduale man mano che si sviluppano le componenti.
In questo caso, i mock vengono limitati a componenti al di sotto del livello di integrazione testato.
I test di integrazione possono avvenire in due diverse direzioni:

- **Top - down**: si verificano prima le componenti con maggior numero di dipendenti, verosimilmente dunque quelle di più alto livello, che dipendono da quelle di livello "più basso".
- **Bottom - up**: si verificano prima le componenti con minor numero di dipendenti, verosimilmente dunque quelle che sono la "base" su cui poggiano le altre componenti.

3.3.4.3 Procedura: test end-to-end di sistema

I test end-to-end verificano il sistema nella sua interezza e simulano l'interazione dell'utente con l'interfaccia grafica. Controllano anche che tutte le componenti funzionino insieme.
Per facilitare l'esecuzione dei test, viene utilizzato il pattern Page Object. Questo permette di separare l'interazione con l'interfaccia in classi dedicate e rende i test più manutenibili. Usando selettori stabili come *data-testid*, non si dipende da etichette o parametri grafici.

3.3.4.3.1 POM pattern per i test E2E

Il pattern Page Object Model (POM) prevede classi dedicate per rappresentare le pagine dell'applicazione. Ogni classe contiene i metodi che l'utente può eseguire sulla pagina, come cliccare un pulsante o inserire testo in un campo.
Il flusso di interazione viene quindi descritto tramite questi metodi.
Inoltre, il POM centralizza la gestione dei selettori in un'unica classe.
Per il tracciamento dei test di sistema è stato creato uno script che, a partire dai file di test e dai codici di test, genera un file con indicazioni sullo stato di copertura e sulle coordinate test.

3.3.4.4 Procedura: test di accettazione (Validazione)

Sono test manuali che vengono eseguiti dalla committente per validare che ciò che è stato sviluppato sia in linea con le attese. Questo tipo di test è guidato dal team, e prevede sia l'organizzazione di un incontro che la messa a disposizione del software da testare. La domanda a cui risponde la validazione e dunque i test di accettazione è "Ho sviluppato il software corretto?", rispetto al ruolo della verifica che invece risponde alla domanda "Ho sviluppato correttamente il software?".

3.3.4.5 Nomenclatura

Viene riportata in seguito la nomenclatura adottata per i test:

T[tipo test]-[progressivo numerico]

Tipo di test:

- **U**: **U**nità
- **I**: **I**ntegrazione
- **S**: **S**ystem
- **A**: **A**ccettazione

lo stato di un test, descritto nel documento: Piano di Qualifica$_{(v2.0.0)}$, segue la seguente specifica:

- **NI**: **N**on **I**mplementato
- **I**: **I**mplementato
- **S**: **S**uperato

3.3.4.6 Strumenti

- **SonarQube:** per l'analisi statica del codice, integrato nel processo di sviluppo.
- **Vitest:** per la definizione e esecuzione dei test di unità e di integrazione
- **Playwright:** per la definizione e esecuzione dei test end-to-end di sistema.

3.3.5 Ruoli

Vengono qui raccolti per praticità i ruoli coinvolti del processo di verifica, con una breve descrizione delle responsabilità di ciascuno.

Ruolo	Specifica
Responsabile di progetto	Stabilisce, pianifica, coordina e monitora le attività di verifica del prodotto.
Verificatore	Garante della qualità dei prodotti, verifica i contenuti, la forma e la struttura dei documenti secondo le norme comuni e specifiche del progetto.

3.4 Processo di validazione

La validazione è un processo soggettivo e orientato all'utente. Verifica che il prodotto sviluppato soddisfi i requisiti della proponente. Valuta l'adeguatezza del prodotto. Serve a capire se il software risponde ai bisogni dell'utente.

3.4.1 Attività: pianificazione della validazione

Questa attività prevede la definizione delle strategie e dei criteri di validazione, nonché la pianificazione delle attività correlate. La pianificazione della validazione avviene contestualmente alla pianificazione dello sviluppo, in modo da garantire che le attività di validazione siano integrate nel ciclo di sviluppo.

3.4.1.1 Procedura: pianificazione della validazione

Per pianificare efficacemente la validazione, il Team segue i seguenti passaggi:

1. Identificazione dei requisiti utente chiave da validare, in collaborazione con la proponente.
2. Definizione dei criteri di accettazione e dei risultati attesi.

3. Assegnazione delle responsabilità per la progettazione e l'esecuzione dei test.

3.4.1.2 Strumenti

- **Jira:** per la gestione delle attività di validazione, con la creazione di task specifici per la pianificazione e l'esecuzione dei test.
- **LaTeX:** per la definizione dei test di accettazione nel documento Piano di Qualifica.
- **Vitest:** per la definizione e esecuzione dei test di unità e di integrazione
- **Playwright:** per la definizione e esecuzione dei test end-to-end di sistema.
- **SonarQube:** per la verifica statica del codice, integrata nel processo di sviluppo.

3.4.2 Attività: definizione dei test di accettazione

In questa attività vengono progettati i casi di test finalizzati a verificare che il sistema soddisfi le esigenze della proponente.

3.4.2.1 Procedura: definizione dei test di accettazione

Per definire i test di accettazione, il Team segue i seguenti passaggi:

1. Raccolta dei requisiti (utente) chiave identificati durante la fase di analisi per comprendere le funzionalità e i comportamenti attesi del sistema.
2. Progettazione dei casi di test di accettazione, specificando le condizioni di test, i dati di input, i risultati attesi e i criteri di successo per ogni caso.
3. Revisione dei casi di test con la proponente per garantire che siano allineati con le aspettative del cliente e che coprano adeguatamente i requisiti chiave.

3.4.2.2 Strumenti

- **Jira:** per la creazione e la gestione delle attività di test di accettazione, con la possibilità di tracciare l'esecuzione e i risultati dei test.
- **Analisi dei Requisiti:** come riferimento per i requisiti da validare.

3.4.3 Attività: esecuzione dei test di accettazione

Questa attività prevede l'esecuzione dei test di accettazione pianificati, la raccolta dei risultati e l'analisi degli esiti per determinare se il prodotto soddisfa i requisiti stabiliti.

3.4.3.1 Procedura: esecuzione dei test di accettazione

Per eseguire i test di accettazione, il Team segue i seguenti passaggi:

1. Preparazione dell'ambiente di test, assicurandosi che sia configurato correttamente e che tutti i componenti necessari siano disponibili per l'esecuzione dei test.
2. Esecuzione dei casi di test di accettazione, seguendo le condizioni e i dati di input specificati, e registrando i risultati ottenuti.
3. Analisi dei risultati dei test, confrontando i risultati ottenuti con quelli attesi per determinare se il prodotto soddisfa i requisiti (utente) e identificare eventuali problemi o aree di miglioramento.

3.4.3.2 Strumenti

- **Jira:** Per tracciare l'avanzamento delle attività di validazione.
- **Piano di Qualifica:** Per la registrazione degli esiti dei test.

3.4.4 Ruoli

Vengono qui raccolti per praticità i ruoli coinvolti del processo di validazione, con una breve descrizione delle responsabilità di ciascuno.

Ruolo	Specifica
Responsabile di progetto	Stabilisce, pianifica, coordina e monitora le attività di validazione del prodotto. È inoltre il membro del gruppo che ha il compito di monitorare il corretto svolgimento delle attività in riferimento ai requisiti e ai test di accettazione stabiliti.
Amministratore	Controlla che il sistema di test sia configurato in modo corretto. L'obiettivo è rendere la validazione più affidabile, ripetibile e automatizzabile.

3.5 Processo di revisione congiunta

Il processo di revisione congiunta è un'attività sincrona tra i membri del gruppo, finalizzata a rivedere insieme sviluppo e documentazione. Questa attività è utile quando si raggiunge una milestone. Gli oggetti della revisione congiunta coincidono spesso con dei Configuration Items. Il processo prevede la partecipazione di un sottoinsieme dei membri del gruppo, scelti in base al loro coinvolgimento nelle attività che hanno contribuito alle modifiche del Configuration Item.

3.5.1 Attività: pianificazione

La pianificazione della revisione congiunta è stabilita dal responsabile in collaborazione con i membri del team che si sono occupati dello sviluppo del Configuration Item. L'attività viene pianificata secondo quanto scritto in §4.1.

3.5.1.1 Procedura: Identificazione dei partecipanti

Il responsabile identifica i membri del team che parteciperanno alla revisione congiunta, tenendo conto dei seguenti criteri:

1. **Coinvolgimento diretto**: priorità ai membri che hanno contribuito direttamente allo sviluppo del Configuration Item.
2. **Competenza tecnica**: includere membri con conoscenze specifiche dell'area di revisione (architetturale, documentale, etc.).

Una volta identificati, i partecipanti vengono formalmente comunicati tramite i canali asincroni.

3.5.1.2 Procedura: Identificazione del materiale oggetto di revisione

Il responsabile, in collaborazione con il team, identifica e documenta il materiale da sottoporre a revisione congiunta:

1. **Determinazione del Configuration Item**: viene identificato il CI (documento, componente software, set di test, etc.) che ha raggiunto una milestone o uno stato di maturità sufficiente per la revisione congiunta.
2. **Comunicazione degli obiettivi di revisione**: vengono stabiliti chiaramente gli obiettivi della revisione.

3.5.1.3 Procedura: Pianificazione della sessione di revisione

Il responsabile pianifica la sessione di revisione congiunta seguendo i seguenti passaggi:

1. **Definizione della data e dell'ora**: viene stabilita una data e un'ora che sia compatibile con la disponibilità di tutti i partecipanti identificati. Si preferisce una sessione sincrona in modo da favorire discussioni approfondite e chiarimenti immediati.
2. **Durata stimata**: viene stimata la durata della sessione in base alla complessità del CI e al numero di partecipanti, con una media di 1-2 ore.
3. **Modalità di conduzione**: viene stabilito il formato della revisione:
    - **In presenza**: se possibile, per favorire una comunicazione più naturale e una collaborazione più stretta.

- **Remota tramite Discord**: utilizzando il server Discord del gruppo con canale dedicato per la sessione.
- **Ibrida**: combinazione di partecipanti in presenza e remoti, utilizzando Discord e uno schermo condiviso.

3.5.1.4 Strumenti

- **Jira**: Per la creazione di task dedicate alle revisioni congiunte e il tracciamento delle attività correlate.
- **Discord**: Per la comunicazione dei dettagli della revisione e la condivisione di aggiornamenti in tempo reale.
- **GitHub**: Per l'accesso al materiale di revisione (documenti, codice, branch).

3.5.2 Attività: esecuzione della revisione congiunta

3.5.2.1 Procedure: Definizione dell'ordine di revisione del materiale

La fase iniziale della revisione prevede una piccola pianificazione dell'ordine di revisione del materiale, che tenga conto della complessità e delle relazioni tra le varie parti che lo compongono. Si lascia la libertà a chi conduce la revisione di stabilire l'ordine e la logica più adatta al caso specifico, tenendo conto dei seguenti criteri:

1. **Logica di sviluppo**: seguire l'ordine in cui le parti sono state sviluppate o documentate, per facilitare la comprensione del processo di sviluppo e delle scelte progettuali.
2. **Relazioni tra le parti**: iniziare con le parti più generali o di alto livello (ad esempio, l'architettura generale) per poi passare a quelle più specifiche (ad esempio, dettagli di implementazione o sezioni di documentazione).
3. **Punti critici**: dare priorità alla revisione di parti che sono state identificate come potenzialmente problematiche o che hanno subito modifiche significative, per garantire che eventuali problemi vengano individuati e risolti tempestivamente.

3.5.2.2 Procedura: revisione del materiale

Durante la revisione ai partecipanti esaminano attentamente il materiale, discutendo eventuali dubbi, criticità o proposte di miglioramento. Ogni problema emerso durante la revisione viene comunicato al responsabile, che si occupa poi di pianificare le attività di risoluzione assieme ai revisori.

3.5.2.3 Strumenti

- **Discord**: Per la comunicazione durante la sessione di revisione, con canale dedicato e possibilità di condividere schermi e documenti in tempo reale.

- **GitHub**: Per l'accesso al materiale di revisione.
- **LiveShare**: Per la condivisione degli ambienti di lavoro e l'editing collaborativo in tempo reale.

3.5.3 Ruoli

Vengono qui raccolti per praticità i ruoli coinvolti del processo di revisione congiunta, con una breve descrizione delle responsabilità di ciascuno.

Ruolo	Specifica
Responsabile di progetto	Coordinatore dell'attività di revisione congiunta
Tutti i membri del gruppo coinvolti nella revisione	Partecipano attivamente alla revisione congiunta, esaminando i materiali e contribuendo alle discussioni
Amministratore	Garantisce l'accesso al materiale soggetto a revisione

3.6 Processo di garanzia della qualità

Il processo di *garanzia della qualità*$_G$ applica in modo sistematico le metriche definite. Coordina anche garanzia della qualità, verifica, validazione e miglioramento. L'obiettivo è mantenere alta la qualità del prodotto finale e dei processi.
Questo processo segue tutte le fasi del ciclo di vita del progetto.

3.6.1 Attività: pianificazione della qualità

3.6.1.1 Procedura: Definizione di metriche e soglie

Il responsabile coordina la pianificazione degli obiettivi di qualità:

1. **Definizione delle metriche di processo**: vengono scelte le metriche chiave per ogni processo. Si privilegiano metriche facili da raccogliere e utili da misurare.
2. **Definizione delle metriche di prodotto**: vengono scelte le metriche chiave per il prodotto finale. Devono restare pertinenti al dominio applicativo.
3. **Definizione delle soglie di accettabilità**: per ogni metrica si fissano soglie minime realistiche e raggiungibili.
4. **Definizione dei valori di ottimalità**: per ogni metrica si fissano anche i valori ideali da raggiungere.

Gli obiettivi di qualità vengono documentati e comunicati a tutto il team nel documento Piano di Qualifica. La definizione delle metriche è presente rispettivamente:

- Per le metriche di processo, nella sezione 5.1.
- Per le metriche di prodotto, nella sezione 5.2.

3.6.1.2 Strumenti

- **Jira**: Per la gestione delle attività di pianificazione della qualità e il tracciamento degli obiettivi.

3.6.2 Attività: monitoraggio continuo

3.6.2.1 Procedura: monitoraggio

Durante lo svolgimento dei periodi, la qualità viene monitorata costantemente attraverso:

1. **Controllo automatizzato:**
   - Le GitHub Actions controllano ogni commit.
2. **Controllo manuale:**
   - I verificatori controllano codice e documenti.
   - Quando serve, fanno test manuali.
3. **Utilizzo del Cruscotto di Qualità:**
   - Le metriche sono visibili nel Piano.
   - Si monitorano trend e scostamenti.

3.6.2.2 Strumenti

- **Github Actions**: Per l'automazione del monitoraggio delle metriche di qualità.
- **Jira**: Per il tracciamento delle metriche di processo.
- **Strumenti di analisi statica**: Per il calcolo automatico di metriche di codice (complessità, coverage)
- **Piano di Qualifica**: valutazione delle metriche tramite il cruscotto di qualità.

3.6.3 Attività: valutazione periodica

La valutazione periodica al fine del miglioramento continuo. Esistono però diverse forme di valutazione periodica.

3.6.3.1 Procedura: review settimanale

Durante le riunioni di team, vengono discusse brevemente le metriche principali e le eventuali criticità emerse. Questa review è scaturita dalla analisi del cruscotto e delle sue metriche.

3.6.3.2 Procedura: review di fine Sprint:

Al termine di ogni Sprint:

- Si confrontano le metriche ottenute con gli obiettivi pianificati
- Si analizzano le cause di eventuali scostamenti significativi
- Si valuta l'*efficacia*$_G$ delle azioni correttive implementate
- Si aggiorna il cruscotto di qualità nel Piano di Qualifica

3.6.3.3 Procedura: review di milestone

In preparazione delle milestone:

- Si effettua una valutazione complessiva della qualità raggiunta
- Si verifica il rispetto di tutti i requisiti di qualità obbligatori

3.6.3.4 Strumenti

- **Jira**: per la raccolta e l'analisi delle metriche di processo.
- **Strumenti di analisi statica**: Per il calcolo automatico di metriche di codice (complessità, coverage)
- **Piano di Qualifica**: per la valutazione delle metriche

3.6.4 Attività: azioni correttive

3.6.4.1 Procedura: Identificazione della deviazione e analisi delle cause

Attraverso il monitoraggio continuo o le review periodiche è possibile identificare eventuali deviazioni dagli standard di qualità imposti.

3.6.4.2 Procedura: Pianificazione dell'intervento

La pianificazione del piano di intervento correttivo si differenzia in in base al grado di importanza, complessità ed estensione delle deviazioni.

- Per deviazioni minori: si assegna una issue correttiva al membro responsabile
- Per deviazioni significative: il responsabile convoca una sessione dedicata per pianificare l'intervento

3.6.4.3 Procedura: Implementazione e verifica

L'azione correttiva viene implementata in modo specifico alla deviazione

3.6.4.4 Linee guida di gestione della qualità

Le seguenti linee guida orientano il Team nella gestione sistematica della qualità:

1. **Prevenzione > Correzione:** Si privilegia l'adozione di pratiche preventive (code review, test automatici, standard di codifica) rispetto alla correzione a posteriori
2. **Automazione quando possibile:** Le verifiche ripetitive vengono automatizzate per ridurre errori umani e liberare tempo per attività a maggior valore aggiunto
3. **Tracciabilità completa:** Ogni decisione relativa alla qualità, ogni deviazione e ogni azione correttiva vengono documentate per garantire tracciabilità
4. **Miglioramento continuo:** Le metriche e le procedure di qualità vengono periodicamente riviste e aggiornate in base all'esperienza accumulata
5. **Responsabilità condivisa:** La qualità è responsabilità di tutti i membri del team, non solo dei verificatori

3.6.4.5 Strumenti

- **Piano di Qualifica**: Documento di riferimento per obiettivi, metriche e soglie di qualità
- **Jira** Per il monitoraggio in tempo reale dello stato delle metriche, il tracciamento delle non conformità, le azioni correttive e la condivisione di *report*$_G$ di qualità con la proponente

3.6.5 Metriche di gestione della qualità

L'efficacia del processo di gestione della qualità viene valutata attraverso la **percentuale di metriche rispettate**. Questa indica il numero di metriche che soddisfano le soglie minime rispetto al totale delle metriche monitorate
Questa meta-metrica viene rendicontata nel Piano di Qualifica insieme alle metriche di processo e prodotto definite nella sezione 5.

3.6.6 Ruoli

Vengono qui raccolti per praticità i ruoli coinvolti del processo di gestione della qualità, con una breve descrizione delle responsabilità di ciascuno.

Ruolo	Specifica
Responsabile	Coordina le attività di gestione della qualità e garantisce il rispetto degli obiettivi definiti nel Piano di Qualifica, verificando periodicamente le metriche riportate a cruscotto
Verificatori	Eseguono le verifiche secondo le procedure definite e misurano (se necessario manualmente) le metriche di qualità di processo e prodotto
Amministratore	Configura e mantiene gli strumenti di automazione per il monitoraggio della qualità (GitHub Actions, Jira)
Tutti i membri	Contribuiscono al mantenimento della qualità seguendo le norme e segnalando deviazioni dagli standard

4 Processi organizzativi

4.1 Processo di gestione

La gestione descrive le modalità di comunicazione interne ed esterne, i vari ruoli e i loro compiti.

4.1.1 Attività: pianificazione

L'attività di pianificazione riguarda per lo più il responsabile, il quale deve adoperarsi per popolare il backlog con attività dettagliate e assegnabili alla singola persona.

4.1.1.1 Procedura: Identificazione

E' necessario prima di ogni cosa capire quali sono le task da svolgere, le dipendenze tra esse, le stime di tempo e risorse umane. L'identificazione avviene durante le riunioni interne di gruppo di progetto, le quali sono documentate in verbali. Ogni decisione presa viene riportata in una tabella dedicata con un riferimento ad una Jira Issue. Vista l'impossibilità e la poca utilità in certi casi di pianificare nel dettaglio ogni singola attività, vengono stabilite in linea generale le macro attività da svolgere. Successivamente il responsabile provvederà a scomporre in task atomiche queste ultime, comunicando all'*amministratore*$_G$ tutte le informazioni necessarie per inserirle in Jira. Le informazioni sono:

- Titolo della task
- Assegnatario
- Ruolo
- Priorità
- Data di scadenza

- Stima in ore
- Data di inizio
- Versione di correzione (RTB o PB)
- Relazioni di dipendenza con altre task

Successivamente l'amministratore provvederà a riportare quanto deciso dal responsabile sulla piattaforma Jira.

4.1.1.2 Strumenti

- **Jira:** Per la gestione del backlog, la creazione e l'assegnazione delle task, e il monitoraggio dello stato di avanzamento.
- **GitHub:** Per la condivisione dei verbali di riunione e dei documenti di pianificazione.
- **Discord:** Per le discussioni interne durante la fase di pianificazione e per il coordinamento del team.

4.1.2 Attività: esecuzione e controllo

L'attività di esecuzione e controllo riguarda tutti i membri del team. Ognuno a questo punto ha compiti chiari e ben definiti da svolgere entro determinate scadenze.

4.1.2.1 Procedura: controllo di processo

Il responsabile durante il periodo di esecuzione si occupa di monitorare lo stato di avanzamento delle task in maniera periodica. Individua eventuali rallentamenti, dipendenze non esaminate o problemi dei componenti del team, e si adopera per risolverli. L'amministratore garantisce al responsabile la continua misura degli indici di qualità di processo definiti in 5.

4.1.2.2 Strumenti

- **Jira:** Per il monitoraggio dello stato di avanzamento delle task, l'identificazione di eventuali blocchi o ritardi, e la gestione delle dipendenze tra le attività.
- **Discord:** Per la comunicazione rapida e il coordinamento del team durante l'esecuzione delle attività, soprattutto in caso di problemi o necessità di riallineamento.
- **Cruscotto di controllo:** per la visualizzazione degli indici di qualità stabiliti.

4.1.3 Attività: retrospettiva

L'attività di retrospettiva riguarda tutti i membri del team e si svolge durante le riunioni interne di gruppo di progetto.

4.1.3.1 Procedura: retrospettiva di periodo

Ad ogni termine di Sprint, il team dedica una porzione della riunione alla retrospettiva. Vengono analizzati i seguenti punti:

- Cosa è andato bene durante lo Sprint: sia a livello organizzativo che esecutivo.
- Cosa è andato male: eventuali problemi riscontrati, difficoltà incontrate, e aree di miglioramento.
- Valutazione degli indici di qualità misurati durante lo Sprint
- Eventuale definizione di azioni correttive o migliorative.

Le decisioni prese durante la retrospettiva vengono documentate nell'apposita sezione dei verbali come definito in 3.1.2. È importante che ogni membro del team partecipi attivamente alla retrospettiva, non limitandosi ad un atteggiamento passivo.

4.1.3.2 Strumenti

- **Cruscotto di controllo:** Per la condivisione dei verbali di retrospettiva e dei materiali di analisi.
- **Discord:** Per la discussione interna durante la retrospettiva e per il coordinamento del team.
- **Jira:** Per la creazione di eventuali task correttive o migliorative emerse durante la retrospettiva.

4.1.4 Ruoli

Vengono qui raccolti per praticità i ruoli coinvolti del processo di gestione, con una breve descrizione delle responsabilità di ciascuno.

Ruolo	Specifica
Responsabile	Gestisce la distribuzione di attività tra i membri, assegnando le attività basandosi sull'ammontare ore rimasto per ogni ruolo ed ogni membro. Si occupa inoltre della redazione dei verbali, della comunicazione con la proponente e dell'aggiornamento del Piano di Progetto. È suo compito il controllo del cruscotto di controllo.
Amministratore	Si occupa del corretto svolgimento del progetto, manutenendo l'infrastruttura, e controllando il corretto aderimento alle pratiche Sprint. Si occupa anche della scrittura delle Norme di Progetto e della creazione delle task su Jira. È suo compito anche il monitoraggio del funzionamento del cruscotto e del recupero automatico delle metriche
Tutti i membri	Sono coinvolti nella retrospettiva di ogni Sprint.

4.2 Processo di infrastruttura

Il processo di Infrastruttura fornisce e mantiene il supporto tecnico, hardware e software, necessario per lo sviluppo e la gestione del progetto.

4.2.1 Attività: implementazione

Per garantire un ambiente di lavoro efficiente, il Team definisce e documenta l'infrastruttura valutando ogni strumento e tecnologia in base ai seguenti vincoli normativi:

- **Funzionalità e Prestazioni:** Lo strumento deve coprire esattamente le necessità del task senza appesantire il flusso di lavoro.
- **Costi e Vincoli di Tempo:** Viene data priorità a soluzioni gratuite, open-source o già fornite in licenza dall'Ateneo, con una curva di apprendimento rapida.
- **Sicurezza e Disponibilità:** I servizi devono garantire alta affidabilità (preferibilmente in Cloud) e permettere una gestione sicura dei permessi e dei dati sensibili (es. credenziali, documenti aziendali).
- **Equipment:** Gli strumenti devono essere multipiattaforma e sufficientemente leggeri da poter essere eseguiti sull'hardware personale di tutti i membri.

Vengono elencati a seguire gli strumenti adottati dal Team per lo svolgimento del progetto.

4.2.1.1 Jira

- **Giustificazione (Funzionalità e Disponibilità):** Scelto per la potenza del suo sistema di tracciamento e la disponibilità in Cloud.

- **Configurazione (Automazioni):** Per ridurre il carico manuale, sono state configurate regole specifiche: ereditarietà delle label padre-figlio, transizione automatica "In Progress" al primo commit, chiusura automatica del parent al completamento dei sub-task, e creazione automatica di task correlati (es. "Scrittura Verbale" alla creazione di "Riunione").

4.2.1.2 VSCode

- **Giustificazione (Costi ed Equipment):** IDE gratuito, leggero e standard de facto, garantisce che tutti i membri operino in un ambiente omogeneo.
- **Configurazione (Estensioni):** Per uniformare lo sviluppo, l'ambiente richiede le estensioni: Atlassian (Jira, Rovo Dev, Bitbucket) per la gestione delle issue integrata, GitHub Pull Requests and Issues, GitHub Actions, e LaTeX Workshop. GitHub Copilot è autorizzato a discrezione del singolo per ottimizzare i tempi di codifica o documentazione.
- **Container:** Per garantire un ambiente di sviluppo omogeneo e facilmente replicabile, è stata configurata una Dev Container con tutte le estensioni e strumenti necessari preinstallati. Questo permette a ogni membro di avviare un ambiente di lavoro coerente con un semplice comando, riducendo i problemi di configurazione locale. L'accesso alla è guidato dall'uso di VSCode Remote Containers, che consente di lavorare direttamente all'interno del container come se fosse un ambiente locale, garantendo così la massima compatibilità e facilità d'uso.

4.2.1.3 LaTeX

- **Giustificazione (Funzionalità e Prestazioni):** Scelto per la sua potenza nella gestione di documenti complessi, garantisce un output professionale ed è ampiamente adottato.
- **Configurazione:** Utilizzo di template standardizzati per i documenti di progetto (es. verbali, report) e integrazione con GitHub per il versionamento.

4.2.1.4 Ecosistema Google (Drive, Presentazioni, Mail)

- **Giustificazione (Sicurezza e Disponibilità):** Garantisce un uptime pressoché totale e permette una gestione centralizzata e sicura degli accessi ai documenti condivisi con l'azienda proponente (Sanmarco Informatica) e i docenti.
- **Configurazione:** Utilizzo obbligatorio dell'account di posta ufficiale del gruppo per le comunicazioni esterne (gestite dal responsabile) per garantire tracciabilità formale. Template standardizzati su Drive per i Diari di Bordo.

4.2.1.5 Canali di Comunicazione (Discord e Whatsapp)

- **Giustificazione (Costi e Prestazioni):** Strumenti a costo zero e ad altissima reattività.
- **Configurazione:** Discord è designato come ambiente di lavoro ufficiale e obbligatorio per le riunioni operative e la collaborazione sincrona. Whatsapp è riservato alle comunicazioni di servizio urgenti.

4.2.2 Attività: acquisizione degli strumenti

La seguente attività espone come replicare e configurare l'infrastruttura di progetto, garantendo che ogni membro possa accedere agli strumenti necessari e che questi siano configurati in modo coerente per supportare le attività di sviluppo e gestione.

4.2.2.1 Procedura: registrazione degli account

Il responsabile, con supporto dell'amministratore, si occupa di:

1. Creare un account per ogni strumento adottato (es. Jira, Google Drive) utilizzando l'email ufficiale del gruppo.
2. Configurare i permessi di accesso in modo che ogni membro abbia i diritti necessari per svolgere le proprie attività (es. permessi di scrittura su Drive, accesso a Jira).
3. Supporta i membri nell'installazione e configurazione degli strumenti locali (es. VSCode con le estensioni necessarie).

4.2.2.2 Procedura: Configurazione degli strumenti

Viene riportato in seguito quanto necessario per configurare gli strumenti adottati replicando la configurazione ottimale per la gestione delle attività di progetto.

4.2.2.2.1 Github

Per la configurazione di GitHub è necessario creare una GitHub Organization, in questo modo tutte le repository di progetto verranno centralizzate e inoltre permette di gestire più facilmente permessi di più persone che vi lavorano contemporaneamente. Successivamente andranno implementate le seguenti configurazioni:

- GitHub Actions: all'interno della directory `.github/workflows` si trovano le definizioni in formato `.yml` delle automazioni che vengono eseguite ad ogni commit o PR.
- Branch Protection: per garantire l'integrità del repository, è stata configurata la protezione del branch `main` richiedendo l'esecuzione con successo delle GitHub Actions. È necessario proteggere i branch di sviluppo "x_in_lavorazione" e "main".

- Email pubbliche: è necessario che ogni membro del team imposti l'email pubblica del proprio account GitHub e che questa sia l'email utilizzata poi anche nella piattaforma Jira. Questo serve per garantire agli smart commit di funzionare correttamente.

4.2.2.2.2 Git

Per la configurazione di *Git*$_G$ è necessario che ogni membro del team configuri il proprio ambiente locale, impostando nome e email con i comandi appositi. Identicamente a quanto detto per GitHub, l'email utilizzata per Git deve essere la stessa dell'account GitHub e di Jira, al fine di garantire il corretto funzionamento degli smart commit.

4.2.2.2.3 Jira

Per configurare lo spazio di lavoro Jira correttamente è necessario configurare diverse funzionalità:

Automazioni:
Il Team ha deciso di adottare alcune automazioni per facilitare la gestione delle attività di progetto. Le automazioni attualmente implementate sono:

- Make child work items inherit labels from parent work items
- When a commit is made → then move issue to in progress
- When all child work items are completed → then close parent
- When all sub-tasks are done → move parent to done
- When Item In Progress → Parent In Progress
- Diario Assegnato → Creo Preparazione Slide
- Riunione Assegnata → Creo Scrittura Verbale

Smart Commit:
E' necessario abilitare gli smart commit dal pannello admin di Jira per collegare le modifiche al codice alle issue Jira. Gli smart commit permettono di automatizzare la transizione delle issue in Jira, migliorando la tracciabilità e l'efficienza del flusso di lavoro. Per esempio, un commit con il messaggio "PROJ-123 #in-progress Implementazione della funzionalità X" sposterà automaticamente l'issue PROJ-123 nello stato "In Progress".

4.2.2.2.4 VSCode

Ambiente di sviluppo integrato (IDE) utilizzato per la scrittura del codice e la gestione della documentazione di progetto. Grazie all'estensione Atlassian per **Jira** e **GitHub**, il Team può integrare direttamente le funzionalità di gestione delle attività e del versionamento all'interno dell'IDE, migliorando l'efficienza del flusso di lavoro e uniformando le

pratiche di sviluppo.
Si presuppone che tutti i membri del gruppo si adattino allo standard comune. Le estensioni attualmente utilizzate sono:

- **Atlassian: Jira, Rovo Dev, Bitbucket**
- **GitHub Pull Requests and Issues**
- **GitHub Actions**
- **GitHub Codespaces**
- **LaTeX Workshop**

Altre estensioni come GitHub Copilot possono essere utilizzate a discrezione del membro del gruppo, al fine di velocizzare, per esempio, il processo di Documentazione 3.1.

4.2.2.2.5 LaTeX

Per l'utilizzo di LaTeXè necessario che ogni membro del gruppo installi i seguenti strumenti:

- **TeX Live**: distribuzione completa di LaTeXche include tutti i pacchetti necessari per la compilazione dei documenti di progetto.
- **VSCode**: IDE per la scrittura dei documenti
- **Latex Workshop**: estensione che fornisce funzionalità avanzate per la scrittura e la compilazione di documenti LaTeXall'interno di VSCode.

4.2.2.2.6 Google Presentazioni

Strumento utilizzato per la creazione delle presentazioni (diari di bordo, ecc...). La scelta di questo strumento è dovuta alla sua facilità d'uso e alla possibilità di collaborare in tempo reale tra i membri del gruppo. Per i Diari di Bordo è stato impostato un template presente nel Google Drive di gruppo.

4.2.2.2.7 Google Drive

Strumento di archiviazione e condivisione dei file con l'azienda proponente e per la condivisione rapida dei Diari di Bordo.
Questo strumento è inoltre utilizzato come mezzo di condivisione dei file con **Sanmarco Informatica**, quali materiali formativi, materiale d'esempio e normative aziendali. Sempre tramite Google Drive vengono condivisi i verbali esterni con l'azienda proponente, in modo da poter essere approvati e archiviati in modo sicuro.

4.2.2.2.8 Gmail

Strumento di comunicazione formale con l'azienda proponente, con i Professori committenti e per la gestione delle restanti comunicazioni ufficiali del gruppo.
Tutti i membri del gruppo devono utilizzare l'account di posta elettronica ufficiale del gruppo per le comunicazioni formali, garantendo così la tracciabilità e la professionalità nelle interazioni esterne.
Le comunicazioni esterne sono gestite dal responsabile salvo diverse indicazioni, il quale deve ricordare che parla a nome di tutto il gruppo.

4.2.2.2.9 Discord

Strumento di comunicazione principale del gruppo, utilizzato per la coordinazione delle attività, la condivisione di informazioni e la collaborazione tra i membri del team.
L'accesso al server Discord del gruppo è obbligatorio per tutti i membri, in quanto rappresenta il canale ufficiale di comunicazione e supporto.

4.2.2.2.10 Whatsapp

Strumento di comunicazione informale e sincrono tra i membri del gruppo, utilizzato per la coordinazione rapida delle attività e la condivisione di informazioni urgenti.
Si raccomanda di essere particolarmente responsivi su questo canale, in quanto spesso viene utilizzato per comunicazioni che richiedono una risposta tempestiva.

4.2.3 Attività: modifiche all'infrastruttura

L'attività di Modifiche all'Infrastruttura riguarda l'implementazione di modifiche agli strumenti e alle tecnologie utilizzate nel progetto, al fine di migliorarne l'efficienza e l'affidabilità.

4.2.3.1 Procedure di Modifica

Le seguenti procedure guidano il Team nell'implementazione delle modifiche all'infrastruttura di progetto, garantendo un uso coerente ed efficiente delle risorse disponibili.

1. **Pianificazione delle modifiche:** viene pianificata l'implementazione delle modifiche, definendo gli obiettivi, le risorse necessarie e i tempi di esecuzione.

2. **Implementazione delle modifiche:** le modifiche vengono implementate secondo la pianificazione stabilita e seguendo le stesse norme descritte in 4.2.1 e 4.2.2.

3. **Verifica delle modifiche:** viene effettuata una verifica delle modifiche per garantire che siano state implementate correttamente e che abbiano raggiunto gli obiettivi prefissati.

4.2.3.2 Strumenti

Gli strumenti adottati per l'implementazione delle modifiche all'infrastruttura di progetto sono i seguenti.

- **Jira:** Utilizzato per tracciare le attività di modifica e monitorare lo stato di avanzamento.
- **GitHub:** Utilizzato per gestire i branch o repository di testing degli strumenti e per la documentazione di questi ultimi.

4.3 Processo di formazione

Il processo di formazione mira a garantire che tutti i membri del gruppo abbiano le competenze e le conoscenze necessarie per svolgere efficacemente le loro attività di progetto.

4.3.1 Attività: pianificazione della formazione

La pianificazione della formazione è un'attività fondamentale per garantire un'efficace trasmissione delle conoscenze. La pianificazione di queste attività segue la pianificazione generale del progetto, in modo da integrare la formazione con le altre attività di progetto.

4.3.1.1 Procedura: Identificazione delle necessità formative

Vengono identificate le necessità formative dei membri del team, in base alle competenze richieste per le attività di progetto. Se queste non possono essere colmate dalle nozioni presenti nelle Norme di Progetto o risultano particolarmente delicate per complessità e/o importanza è necessaria una pianificazione approfondita.

4.3.1.2 Procedura: Pianificazione delle attività formative

Vengono pianificate le attività formative, definendo gli obiettivi, i contenuti, le risorse necessarie e i tempi di erogazione. Queste attività formative devono svolgersi il prima possibile per garantire la continuazione delle attività produttive.

4.3.1.3 Strumenti

- **Jira:** Utilizzato per tracciare le attività di modifica e monitorare lo stato di avanzamento.

4.3.2 Attività: erogazione della formazione

L'erogazione della formazione si implementa attraverso sessioni di formazione. In particolare, sono identificabili due tipologie di sessioni:

4.3.2.1 Procedura: Formazione Autonoma

Sessioni individuali di studio e pratica, al fine di comprendere il funzionamento e le strategie per l'utilizzo ottimale di uno strumento.
La formazione risulta completa quando è dimostrabile concretamente con l'utilizzo dello strumento.

4.3.2.2 Procedura: confronto

Sessioni di formazione condotte da tutti i membri coinvolti su un argomento specifico, con l'obiettivo di condividere conoscenze, best practice e formulare soluzioni comuni. Utili soprattutto per quegli strumenti che necessitano di gestioni particolarmente attente, poco intuitive o particolarmente vasti come funzionalità.

4.3.2.3 Strumenti

- **Jira:** Per tracciare le attività di formazione e monitorare lo stato di avanzamento.
- **Discord:** Per la conduzione delle sessioni di confronto e per la comunicazione tra i membri coinvolti nella formazione.
- **Google Drive:** Per la condivisione dei materiali formativi e delle registrazioni delle sessioni di formazione.

4.3.3 Attività: valutazione della formazione

L'attività di Valutazione della Formazione serve a verificare l'efficacia delle sessioni di formazione e l'acquisizione delle competenze da parte dei partecipanti.

4.3.3.1 Procedura: verifica pratica

Una volta terminata la formazione vengono assegnate task che richiedono l'applicazione delle conoscenze apprese. Nel caso in cui questa attività faccia emergere dubbi sarà quindi necessaria una sessione di formazione e confronto con i membri del team che abbiamo una visione più chiara e possano indirizzare in modo mirato verso una più completa comprensione.

4.3.3.2 Strumenti

Gli strumenti utilizzati per la valutazione della formazione includono:

- **Jira:** Per tracciare i task di verifica assegnati post-formazione.
- **Discord:** Per la discussione e il confronto tra i membri coinvolti nella formazione.
- **VSCode:** IDE principale del gruppo che grazie alle integrazioni con Jira e GitHub permette una gestione semplice della procedura di Verifica pratica.

4.3.4 Ruoli

Vengono qui raccolti per praticità i ruoli coinvolti del processo di formazione, con una breve descrizione delle responsabilità di ciascuno.

Ruolo	Specifica
Responsabile	Coordina le attività di gestione della formazione organizzando le sessioni e monitorando il corretto svolgimento degli incontri formativi.
Tutti i membri	Partecipano attivamente alle sessioni di formazione e contribuiscono alla valutazione dell'efficacia della formazione. È inoltre utile segnalare eventuali problematiche o dubbi.

5 Metriche della qualità

Lo standard ISO/IEC 9126-1:1999 individua tre categorie fondamentali di requisiti per la qualità che concorrono alla creazione del prodotto e interconnesse tra loro secondo il Modello a V(Figura 5) (*V-Model*$_G$):

- **Bisogni utente**: sono i requisiti che il prodotto deve soddisfare per incontrare i bisogni dell'utente. L'output corrispondente e' la qualità in uso, verificata da apposite metriche esterne.
- **Requisiti di qualità esterni**: corrispondono ai requisiti che riguardano le caratteristiche proprie del prodotto da realizzare. Il loro soddisfacimento definisce la qualità esterna del prodotto, validata anch'essa da opportune metriche esterne.
- **Requisiti di qualità interni**: sono i requisiti legati strettamente ai processi di sviluppo software che quando rispettati risultano in codice di qualità, processi efficaci e organizzazione efficiente.

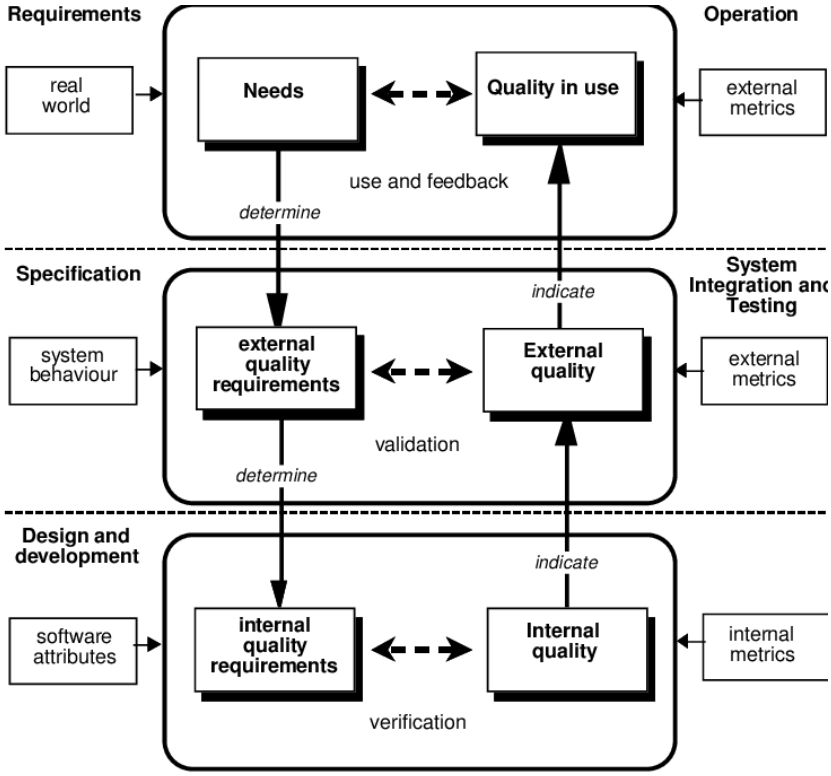

Figura 4: Modello a V

Di seguito vengono indicate le metriche utilizzate per valutare la qualità del prodotto e dei processi di sviluppo.

5.1 Qualità di Processo

In questa sezione vengono definite le metriche utilizzate per valutare l'efficienza e l'efficacia dei processi del ciclo di vita del software, in conformità con lo standard ISO/IEC 12207.

5.1.1 Processi primari

5.1.1.1 Processo di fornitura

MPC-1 - Earned Value (EV)

Descrizione: Valore del lavoro effettivamente completato in un determinato momento, espresso in termini di budget approvato.

Formula di calcolo:

$$EV = \sum_i (\%\text{Completamento}_i \times \text{Budget}_i)$$

MPC-2 - Planned Value (PV)

Descrizione: Valore del lavoro che si era pianificato di completare entro una certa data.

Formula di calcolo:

$$PV = \sum_i (\%\text{Pianificato}_i \times \text{Budget}_i)$$

MPC-3 - Actual Cost (AC)

Descrizione: Costo realmente sostenuto per il lavoro svolto fino alla data corrente.

Formula di calcolo:

$$AC = \sum \text{Costi Sostenuti}$$

MPC-4 - Cost Performance Index (CPI)

Descrizione: Indice di efficienza dei costi. Un valore < 1 indica che il progetto è fuori budget.

Formula di calcolo:

$$CPI = \frac{EV}{AC}$$

MPC-5 - Schedule Performance Index (SPI)

Descrizione: Indice di efficienza della pianificazione. Un valore < 1 indica che il progetto è in ritardo.

Formula di calcolo:

$$SPI = \frac{EV}{PV}$$

MPC-6 - Estimate to Complete

Descrizione: Rappresenta l'ammontare rimanente stimato per completare il progetto

Formula di calcolo:

$$ETC = EAC - AC$$

MPC-7 - Estimate at Completion (EAC)

Descrizione: Stima totale dei costi alla fine del progetto, basata sulle performance attuali.

Formula di calcolo:

$$EAC = \frac{BAC}{CPI}$$

5.1.1.2 Processo di Sviluppo

MPC-8 - Deployment Frequency

Descrizione: Frequenza con cui il team rilascia nuove versioni del software, potenzialmente trasportabili in produzione.

Formula di calcolo:

$$DF = \frac{\text{Numero di deployment}}{\text{Periodo di tempo}}$$

MPC-9 - Requirements Stability Index

Descrizione: Rappresenta la stabilità dei requisiti del prodotto software nel tempo.

Formula di calcolo:

$$RSI = \frac{\#\text{Req originali} - (\#\text{Req modificati} + \#\text{Req eliminati} + \#\text{Req aggiunti})}{\#\text{Req originali}} \cdot 100$$

MPC-10 - Average Build Time

Descrizione: Tempo medio impiegato per completare il processo di build e test automatici.

Formula di calcolo:

$$T_{avg} = \frac{\sum_{i=1}^{n} T_{build_i}}{n}$$

MPC-11 - Sprint Velocity

Descrizione: Quantità di lavoro in uno Sprint, considerando le aggiunte, gli elementi non completati, e le modifiche al backlog.

Formula di calcolo:

$$SV = \#\text{Workitem completati} - (\#\text{Workitem aggiunti} + \#\text{Workitem non completati})$$

MPC-12 - Burndown Adherence

Descrizione: Rapporto tra il coefficiente di elementi completati e il coefficiente ideale. Questo indice ideale va dal numero di workitem nel momento dell'apertura dello Sprint a 0 al termine dello Sprint.

Formula di calcolo:

$$BA = \frac{\frac{d\text{Consumo reale}}{dx}}{\frac{d\text{Consumo ideale}}{dx}}$$

5.1.1.3 Processo di Documentazione

MPC-13 - Indice di Gulpease

Descrizione: Indice che valuta la leggibilità di un testo in lingua italiana. Valori bassi indicano bassa leggibilità.

Formula di calcolo:

$$G = 89 + \frac{300 \times (\#\text{Frasi}) - 10 \times (\#\text{Lettere})}{\#\text{Parole}}$$

MPC-14 - Correttezza Ortografica

Descrizione: Misura la densità di errori ortografici presenti nella documentazione.

Formula di calcolo:

$$C = \#\text{errori ortografici}$$

5.1.1.4 Processo di Verifica

MPC-15 - Code review turnaround time

Descrizione: Tempo medio impiegato per completare una revisione del codice in vista di una Pull Request.

Formula di calcolo:

$$CRT = \frac{\sum_{i=1}^{n} T_{review_i}}{n}$$

T_{review_i} tempo impiegato per completare la revisione della PR i.

MPC-16 - Test success rate

Descrizione: Percentuale di test superati rispetto al totale dei test eseguiti.

Formula di calcolo:

$$TSR = \left(\frac{\#\text{Test superati}}{\#\text{Test eseguiti}}\right) \times 100$$

MPC-17 - Code Coverage

Descrizione: Percentuale di codice coperta da test

Formula di calcolo:

$$CC = \frac{\#\text{LOC testate}}{\#\text{LOC totali}} \times 100$$

5.1.2 Processi Organizzativi

5.1.2.1 Processo di Gestione dei Rischi

MPC-18 - Rischi non previsti

Descrizione: Numero di rischi non previsti ad inizio periodo, per ciascun periodo di progetto.

Formula di calcolo:

$$RNP_i = \#\text{Rischi non previsti nel periodo } i$$

5.1.2.2 Processo di Gestione della qualità

MPC-19 - Metriche soddisfatte

Descrizione: Percentuale di metriche di qualità che vengono soddisfatte in un determinato istante di tempo.

Formula di calcolo:

$$MS = \left(\frac{\#\text{Metriche soddisfatte}}{\#\text{Metriche totali}}\right) \times 100$$

5.2 Qualità di Prodotto

In questa sezione vengono definite le metriche per valutare la qualità intrinseca del prodotto software, basandosi sul modello ISO/IEC 9126.

5.2.1 Funzionalità

MPD-1 - *Requisiti obbligatori*$_G$ soddisfatti

Descrizione: Percentuale di requisiti funzionali obbligatori implementati e testati.

Formula di calcolo:

$$R_{\text{ob}} = \left(\frac{\text{Requisiti obbligatori soddisfatti}}{\text{Requisiti obbligatori totali}}\right) \times 100$$

MPD-2 - *Requisiti opzionali*$_G$ soddisfatti

Descrizione: Percentuale di requisiti funzionali opzionali implementati e testati.

Formula di calcolo:

$$R_{\text{op}} = \left(\frac{\text{Requisiti opzionali soddisfatti}}{\text{Requisiti opzionali totali}}\right) \times 100$$

MPD-3 - *Requisiti desiderabili*$_G$ soddisfatti

Descrizione: Percentuale di requisiti funzionali desiderabili implementati e testati.

Formula di calcolo:

$$R_{\text{de}} = \left(\frac{\text{Requisiti desiderabili soddisfatti}}{\text{Requisiti desiderabili totali}}\right) \times 100$$

5.2.2 Affidabilità

MPD-4 - Broken links

Descrizione: Numero di riferimenti ipertestuali all'interno dell'applicazione che non portano alla risorsa corretta o non portano ad alcuna risorsa.

Formula di calcolo:

$$BL = \#\text{Broken Links}$$

MPD-5 - Branch Coverage

Descrizione: Percentuale di rami decisionali (if, switch, loop) percorsi durante i test.

Formula di calcolo:

$$BC = \left(\frac{\text{Rami testati}}{\text{Rami totali}}\right) \times 100$$

MPD-6 - Statement Coverage

Descrizione: Percentuale di istruzioni (statements) del codice sorgente eseguite durante i test automatici.

Formula di calcolo:

$$SC = \left(\frac{\text{Istruzioni testate}}{\text{Istruzioni totali}}\right) \times 100$$

5.2.3 Usabilità

MPD-7 - Profondità di Navigazione

Descrizione: Numero massimo di click necessari per raggiungere una qualsiasi pagina di contenuto partendo dalla vista principale.

Formula di calcolo:

$$\max_{i \in I} \text{Click}_i \qquad \begin{array}{l}(\text{dove } I = \text{insieme delle pagine}) \\ (\text{dove Click}_i = \text{numero di click per raggiungere la pagina } i)\end{array}$$

5.2.4 Efficienza

MPD-8 - Indexing time

Descrizione: Tempo di *indicizzazione*$_G$ per un *DiP*$_G$ di dimensioni standard (1-4GB).

Formula di calcolo:

$$IT = \text{Tempo di indicizzazione (secondi)}$$

MPD-9 - Search Time

Descrizione: Tempo medio di risposta per un set di *query*$_G$ predefinite.

Formula di calcolo:

$$ST = \sum_{i=1}^{n} \frac{\text{Tempo di risposta}_i}{n} \quad \begin{array}{l}(\text{dove } n = \text{numero di query}) \\ (\text{dove } i = \text{indice della query})\end{array}$$

MPD-10 - Average CPU usage

Descrizione: Percentuale media di utilizzo della CPU durante l'esecuzione del software. Si considera un campione di misurazioni effettuate in condizioni di carico (durante l'indicizzazione o la ricerca) nonché in idle. Quest'ultimo per assicurare che il software non consumi risorse inutilmente quando non è in uso.

Formula di calcolo:

$$CPU_{avg} = \sum_{i=1}^{n} \frac{\text{Utilizzo CPU}_i}{n} \quad (\text{dove } n = \text{numero di misurazioni})$$

MPD-11 - Peak memory usage

Descrizione: Massima quantità di memoria RAM utilizzata durante l'esecuzione del software. Permette di individuare eventuale scarsa gestione delle risorse.

Formula di calcolo:

$$MEM_{peak} = \max(\text{Utilizzo RAM}_i) \quad (\text{dove } i = 1, \ldots, n, \text{istante di tempo } i)$$

5.2.5 Manutenibilità

MPD-12 - Complessità Ciclomatica

Descrizione: Misura la complessità del flusso di controllo del codice. Valori superiori a 15 indicano codice difficile da testare e manutenere.

Formula di calcolo:

$$M = E - N + 2P \quad (\text{dove } E = \text{archi}, N = \text{nodi}, P = \text{componenti connessi})$$

MPD-13 - Accoppiamento tra Classi (CBO)

Descrizione: Numero di classi a cui una determinata classe è accoppiata (usa o è usata da). Un valore alto indica scarsa modularità.

Formula di calcolo:

$$CBO = \frac{\#\text{Dipendenze}}{\#\text{Componenti}}$$

MPD-14 - Code smells

Descrizione: Numero di code smells, misurato ogni 1000 righe di codice (KLOC). I code smells sono indicatori di potenziali problemi di design o implementazione.

5.2.6 Portabilità

MPD-15 - SO Compatibility

Descrizione: Numero di sistemi operativi supportati dal software. Un valore alto indica una maggiore portabilità.

Formula di calcolo:

$$SO_{comp} = \#\text{Sistemi operativi supportati}$$

5.3 Strumenti

5.3.1 Jira Software

Utilizzato come strumento di Project Management per il tracciamento dei requisiti e la gestione del workflow.

- **Metriche associate:** MPC-1 ... MPC-7 (Earned Value Management), MPC-17 (Metriche soddisfatte), MPD-1 ... MPD-3 (Requisiti).
- **Utilizzo:** Attraverso la funzione di esportazione $JQL_G \rightarrow$ csv, e uno script python appositamente sviluppato, vengono estratti i dati necessari per il calcolo delle metriche di Earned Value e dei requisiti soddisfatti, permettendo un monitoraggio costante dell'andamento del progetto.

5.3.2 GitHub & GitHub Actions

Viene utilizzato per il controllo di versione e l'automazione della CI/CD.

- **Metriche associate:** MPC-8 (Deployment Frequency), MPC-10 (Avg Build Time), MPC-13 (Code Review Turnaround Time), MPC-14 (Test success rate).
- **Utilizzo:** I workflow di GitHub Actions oltre che garantire la metrica MPC-14 (Test success rate) attraverso l'esecuzione di test automatici, permettono di tracciare i tempi di build e code review, fornendo dati utili per il calcolo delle metriche di processo.

5.3.3 Strumenti di Analisi Statica e Qualità del Codice

5.3.3.1 SonarQube

Piattaforma centrale per la *Continuous Inspection*$_G$ del codice TypeScript (Angular) e JavaScript (Electron).

- **Metriche associate:** MPD-6 (Statement Coverage), MPD-5 (Branch Coverage), MPD-12 (Complessità Ciclomatica), MPD-13 (CBO), MPD-14 (Code Smells).
- **Utilizzo:** Analizza staticamente il codice ad ogni *Push*$_G$, verificando che il superamento dei *Quality Gates*$_G$ stabiliti sia coerente con le soglie definite nelle metriche di manutenibilità.

5.3.3.2 Aspell & Script Python (Custom)

Data la natura della documentazione in lingua italiana, si rende necessario uno strumento specifico per l'analisi testuale.

- **Metriche associate:** MPC-11 (Indice di Gulpease), MPC-12 (Correttezza Ortografica).
- **Utilizzo:** Uno script Python che utilizza la libreria `textstat` per il calcolo dell'indice di Gulpease e le API di *Aspell* per il conteggio degli errori ortografici nei file Markdown/*LaTeX*$_G$.

5.3.4 Strumenti di Testing e Performance (Electron-Angular)

5.3.4.1 Playwright

Framework di testing End-to-End utilizzato per simulare l'interazione utente all'interno dell'ambiente Electron.

- **Metriche associate:** MPD-7 (Profondità di Navigazione).
- **Utilizzo:** Viene configurato un *crawler*$_G$ automatizzato che percorre l'albero di navigazione dell'applicazione Angular, validando la raggiungibilità dei link e calcolando il numero di interazioni (click) necessarie per raggiungere i nodi foglia.

5.3.4.2 Electron Manager & Chrome DevTools Protocol

Per misurare l'efficienza di un'applicazione Electron, è necessario monitorare l'istanza di Chromium sottostante.

- **Metriche associate:** MPD-10 (Average CPU usage), MPD-11 (Peak memory usage).
- **Utilizzo:** Durante le sessioni di test automatizzati con Playwright, vengono invocati script Node.js che interrogano le API `process.getProcessMemoryInfo()` e `process.getCPUUsage()` di Electron per campionare il consumo di risorse in idle e sotto carico.

5.3.4.3 TotalValidator

Integrato nei tool di sviluppo, viene utilizzato per profilare i tempi di risposta dell'interfaccia Angular.

- **Metriche associate:** MPD-4 (Broken links).
- **Utilizzo:** Controllo formale di consistenza e correttezza della struttura dell'interfaccia utente Angular.

5.3.4.4 Funzioni interne di misurazione performance

Integrate direttamente nel codice sorgente, queste funzioni permettono di misurare i tempi di indicizzazione e risposta alle query, fornendo dati precisi per le metriche MPD-8 (Indexing time) e MPD-9 (Search Time).

7-ZPUs